DAMAGE-STAGES — our damage formula against the authority, stage by stage

Version: 5.261.0 — 2026-09-06.

5.260.0 - NOTHING IN THE DAMAGE CHAIN MOVED, AND THE DIFFERENTIAL THAT WOULD SAY SO WAS RE-RUN RATHER THAN ASSUMED. All four items in this version are ANNOUNCEMENT and REFUSAL-ORDER defects: a missing `-activate` above a move line, an item announcement sitting above the hit steps instead of inside them, `mustrecharge` 's position in the BeforeMove chain, and a missing `-fail` for a move with nothing to aim at. No stage, no multiplier and no truncation position in this document is affected. `tests/test-engine-diff.js --n 6000 --seed 20260804` was re-run on the settled release `a985300cb8ed` and reads **0 disagreements at the midpoint and 0 at every one of the sixteen roll indices**, which is the reading this page exists to carry.

ONE OF THE FOUR SITS ADJACENT TO THIS CHAIN AND IS WORTH NAMING FOR THAT REASON. The item announcement moved into the per-target STEP LIST, beside the step that breaks screens — both are a move's own `onTryHit` , which the authority fires from inside `spreadMoveHit` at the top of the hit loop. The step list is the structure this page's ordering claims rest on, and the change adds an entry to it without moving any existing one.

5.259.0 - NOTHING IN THE DAMAGE CHAIN MOVED. This version is a documentation correction to two published figures elsewhere — a mis-cited quality-filter statistic and a quarantined leaf calibration figure. No stage, no multiplier and no ordering in this document is affected, and the header moves only because the project version does.

5.258.0 - NO DAMAGE STAGE MOVED AND NO MULTIPLIER VALUE CHANGED, BUT TWO OF TONIGHT'S FIVE ITEMS SIT CLOSE ENOUGH TO THIS CHAIN THAT SAYING SO IS THE POINT OF THE BLOCK. Nothing in this document's stage table, its multiplier classes or its ordering changes. **FAIRY AURA IS A BASE-POWER MODIFIER AND ITS MULTIPLIER WAS ALWAYS CORRECT** - what was wrong was the FIELD it read. `field.aura` had two writers where the identically-shaped `recomputeWeatherSuppression` has four, so a holder that had left, returned or been killed went on pricing Fairy moves at a value that was itself right. **A stage table cannot see that defect and would have called this engine clean**, because the stage is asked *what does this multiplier do* and the defect was *is this multiplier present*. **BIG ROOT IS AN ORDERING FACT ON THE HEAL CHAIN, NOT THE DAMAGE CHAIN**, and it is the same shape one road over: `Battle#heal` truncates the base value BEFORE it runs `TryHeal` , so `trunc(155/16) = 9` then `modify(9, 5324, 4096) = 12` , while folding the multiplier into the fraction reaches 12 by luck and truncating after a float multiply reaches 11. **Truncation position is the thing this page exists to pin, and it was wrong on a chain this page does not cover.**

AND THE DAMAGE ENDPOINTS WERE THE VISIBLE HALF OF TONIGHT'S INSTRUMENT DEFECT, WHICH IS A CLAIM ABOUT THE HARNESS AND NOT ABOUT THE FORMULA. Two of the four failures of `tests/test-game-differential.js` were its damage-endpoint clauses, and all four were one line: `const PRIMARY_ARM = ARMS[0]` bound three module-scope staged pins to whatever sat first in the arm list, and `ARMS[0]` stopped being the max-damage arm on 2026-08-13 when the opt-in `middle` arm was prepended. **The engine was never implicated and that was measured rather than asserted**: a 14-call control held this engine's damage interior constant at `108..127` while the authority's wandered and on one run lost its own minimum, and once the pins were bound BY NAME the authority's span came DOWN onto this engine's - knock-off `108..177` to `108..127` , contact punish `66..104` to `66..78` - with this engine's own numbers never moving in either scenario. **The two damage tables are exactly equal on all sixteen rolls whenever the harness stops critting on one side only.** A damage clause that was green for fourteen days because a hash happened to land favourably is not evidence about the formula, and this page should be the last to read it as such.

THE STAGE-BY-STAGE INSTRUMENT WAS NOT RE-RUN IN THIS PASS AND IS STILL OWED, AND THE ENGINE MOVED AGAIN UNDER IT. `engine/medicham2-browser.js` was edited twice tonight, for Fairy Aura and for Beat Up's ally order. Neither touches a multiplier value and neither is on this chain, and neither of those facts discharges the debt: **a stage table is a PINNED measurement, the release it was pinned to is superseded again, and the correct response is a re-run rather than an argument that the fixes cannot have reached a multiplier.** The stage artifact also still predates the instrument digest introduced one version ago, so it cannot say which bytes of the harness produced it. **An unstamped artifact is a figure to WITHHOLD and re-measure, never one to argue about.**

WHICH ARTIFACT HOLDS WHICH WHOLE-GAME FIGURE, AND IT IS NOT EVIDENCE ABOUT THE DAMAGE CHAIN. `data/game-differential.json` was republished off a settled tree this pass and holds board-material 50 of 961 with protocol first-divergence 151 of 961, on release `db248fe67a5e` at cap 20 with the empirical arm and the frozen pool. There is no longer a correctly-measured value of that quantity sitting outside the published artifact. **A whole-game count says nothing about the formula:** a board parts for a targeting error, a status, a switch, a faint or a stale field just as readily as for a damage roll, and a clean stage table is evidence about the damage chain and about nothing else. The damage differential itself is unchanged at **0 of 6000** disagreements at the midpoint and at all sixteen corners, seed 20260804.

5.257.0 - NO DAMAGE STAGE MOVED, NO MULTIPLIER CHANGED, AND NO DAMAGE VALUE WAS RE-DERIVED IN THIS PASS. Nothing in this document's stage table, its multiplier classes or its ordering changes. This version touched a DRIVER rule, the comparator's leaf set and the instrument's own digest - the stand-in player's candidate list, which fourteen more pieces of hidden state are compared, and whether a run can say which bytes measured it. **A LEAF IS STATE, NOT A MULTIPLIER**, so widening the comparison from 40 leaves to 54 adds nothing to this chain and removes nothing from it. The one figure on this page that a reader might be tempted to attach to the damage chain - board-material 34 to 47 on the paired empirical run - is a count of games whose BOARDS parted, and a board parts for a targeting error, a status, a switch or a faint just as readily as for a damage roll.

THE STAGE-BY-STAGE INSTRUMENT WAS NOT RE-RUN IN THIS PASS, AND IT IS STILL OWED FROM THE VERSION BELOW. This version's changelog records no run of the stage-by-stage comparison, so the last measured stage result remains the one recorded in the 5.254.0 block below, on the release named there. It is carried forward and is not restated here as a fresh measurement. The 5.256.0 block below already recorded that this measurement had gone from merely old to OWED, because `medicham2-browser.js` - the file this page's instrument compares against the authority - moved under it. Nothing in this version discharges that. **A stage table is a PINNED measurement and the release it was pinned to is superseded; the correct response is a re-run, not an argument that a driver fix cannot have touched a multiplier.**

AND THIS VERSION ADDS A REASON TO RE-RUN IT THAT THIS PAGE DID NOT PREVIOUSLY HAVE: THE INSTRUMENT IS NOW DIGESTED, AND THE STAGE ARTIFACT PREDATES THAT STAMP. A measurement here pins the engine release, the census and the team pool. All three are INPUTS. Until this version nothing recorded the bytes of the code that READS them, and `engine/arms_comparable.js` was measured answering "COMPARABLE" about two runs that were playing different driver code. `engine/steering.js` now digests the instrument's require closure into

`steering.driver_code` and `engine/game_differential.js` voids a run whose instrument moved while it played. The stage-by-stage artifact carries no such stamp, because it was written before the stamp existed - so it cannot say which instrument produced it. That is a reason to re-take the measurement, and it is not a reason to doubt the stage table itself: **an unstamped artifact is a figure to WITHHOLD and re-measure, never one to argue about.**

WHICH ARTIFACT HOLDS WHICH WHOLE-GAME FIGURE, WITH THE ARM NAMED, AND NEITHER IS EVIDENCE ABOUT THE DAMAGE CHAIN. `data/game-differential.json` was again NOT republished and still holds board-material 46 of 961; that is the figure the gate prints and it is stale as a description of tonight's engine. Tonight's runs live in the verification artifacts under `data/verification/` , one per arm: the empirical arm reads 147 protocol and 55 board-material on the repaired driver, and the joint arm's 53 stands and repeats. **Two corrections carried onto this page for completeness, because both were published tonight and both touch how a figure on any page is read:** the joint arm was never non-deterministic - six runs split cleanly either side of the driver fix and each group is bit-identical inside itself - and the empirical figure of 47 is the pre-widening comparator's reading of the same run that the widened comparator reads as 55.

5.256.0 - NO DAMAGE STAGE MOVED, NO MULTIPLIER CHANGED, AND NO DAMAGE VALUE WAS RE-DERIVED IN THIS PASS. Nothing in this document's stage table, its multiplier classes or its ordering changes. Two defects landed in the simulator and neither is on this chain. **WHICH BODY A HIT LANDS ON IS NOT THE DAMAGE CHAIN, AND THE DISTINCTION IS WORTH STATING ON THIS PAGE RATHER THAN ASSUMED.** The headline defect is a targeting error: the release turn of a two-turn move aimed at the lowest live enemy index instead of replaying the aim stored when the move was charged, so a Phantom Force charged at slot b struck slot a. Every multiplier on this page then applied correctly - to the wrong body. This chain computes what a hit does; it does not decide where the hit goes, and a page that only checks the chain would call that engine clean. The second defect is a charge wrapper surviving a BeforeMove refusal, which is a legality question and not a damage one, and it fires 2 times in 961 games.

THE STAGE-BY-STAGE INSTRUMENT WAS NOT RE-RUN IN THIS PASS, AND THE SIMULATOR MOVED UNDER IT. This version's changelog records no run of the stage-by-stage comparison, so the last measured stage result is the one recorded in the 5.254.0 block below, on the release named there. It is carried forward and is not restated here as a fresh measurement. **And it is now owed again rather than merely old:** `engine/engine_release.js drift` reports that `medicham2-browser.js` - the file this page's instrument compares against the authority - is the one source that moved between this version's releases. A stage table is a PINNED measurement, and the release it was pinned to is superseded. The correct response to that is a re-run, not an argument that a targeting fix cannot have touched a multiplier.

WHICH ARTIFACT HOLDS WHICH WHOLE-GAME FIGURE, WITH THE ARM NAMED, AND NEITHER IS EVIDENCE ABOUT THE DAMAGE CHAIN. `data/game-differential.json` was again NOT republished and still holds board-material 46 of 961; that published figure is stale as a description of tonight's engine and is what the gate prints. The measurements taken on tonight's engine live in this version's verification artifacts under `data/verification/` , one per driver arm, and they may not be paired with each other: the joint arm reads board-material 53 of 961 with protocol first-divergence 138, and the empirical arm reads 34 with protocol 121. Name the artifact and the arm whenever any of them is quoted. A whole-game figure says nothing about the formula, and a clean stage table is evidence about the damage chain and about nothing else.

5.255.0 - NO DAMAGE STAGE MOVED, NO MULTIPLIER CHANGED, AND NO DAMAGE VALUE WAS RE-DERIVED IN THIS PASS. Nothing in this document's stage table, its multiplier classes or its ordering changes. Two mechanics landed in the simulator and neither is on this chain: Imprison is a legality seal that decides whether a move is played at all, and a bounced Parting Shot is a pivot and a stat drop. Three more comparator leaves were wired and a leaf is state, not a multiplier. **This version's changelog records no re-run of the stage-by-stage instrument, so the last measured stage result is the one recorded in the 5.254.0 block below, on the release named there.** It is carried forward and is not restated as a fresh measurement here; a release cut after it stales it by construction, and the paragraph below is about exactly that.

A RELEASE DIGEST THAT MOVES NOW SAYS WHY, AND THIS PAGE IS ONE OF THE PLACES THAT COSTS THE MOST. The stage table on this page is a PINNED measurement: it is quoted against a frozen release, and a new release id makes it owed again. That is correct when the sources really changed and expensive when they did not. In this version a line-ending change to one source moved a release digest while all 26 frozen sources stayed content-identical, and five heavy clauses were re-run before anybody knew that. `engine/pin_guard.js` now reports each drift as CONTENT-CHANGED or EOL-ONLY, so the next reader of this page can tell a stage table that is genuinely stale from one that only looks stale. **Nothing is excused by this.** The digest still hashes raw bytes, the id still moves, and a stage figure measured against a superseded release is still owed a re-run. The two failing clauses of this version are diagnosed CONTENT-CHANGED and still fail, with their counts withheld.

WHICH ARTIFACT HOLDS WHICH WHOLE-GAME FIGURE, AND NEITHER IS EVIDENCE ABOUT THE DAMAGE CHAIN. `data/game-differential.json` was again NOT republished and still holds board-material 46 of 961. That published figure is stale as a description of tonight's engine; the measurement taken on tonight's engine is board-material 35 of 961 with protocol first-divergence 120, and it lives in this version's verification artifact under `data/verification/` . Name the artifact whenever either is quoted. A whole-game figure says nothing about the formula: a clean stage table is evidence about the damage chain and about nothing else.

THE DAMAGE ROWS IN THE REMAINING WHOLE-GAME LEAVES ARE FENCED, AND THAT IS A STATEMENT ABOUT THE POPULATION AND NOT ABOUT THE FORMULA. Of the 37 whole-game rows standing at the start of this version, roughly 12 are damage-VALUE rows already fenced by a filed row — two engines drawing a different number out of the same legal damage range, which the stage instrument is deliberately built not to compare, because it compares the CHAIN and not the roll. Two more are Poison Touch and are the same shape: a die value, with the ability wired and both engines reaching the draw at the same point. **A die that lands differently is not a formula that computes differently, and this page has no business claiming either way about it.**

5.254.0 - NO DAMAGE STAGE MOVED AND NO MULTIPLIER CHANGED, AND THE STAGE-BY-STAGE INSTRUMENT WAS RE-RUN AFTER EVERY RELEASE THIS VERSION CUT: `data/engine-diff.json` **READS 6,000 COMPARED, 0 DISAGREED, ON RELEASE** `ae608567e8a8` **.** Nothing in this document's stage table, its multiplier classes or its ordering changes. Four mechanics landed in the simulator and none of them is on this chain: self-inflicted stat drops through a broken Substitute are stat STAGES rather than damage, a sleep attribution is narration, and the sun refusing a freeze is a status immunity. The scope caveat is unchanged and is stated rather than dropped - 134 comparisons skipped as multi-hit, 0 non-finite. **So the stage table below is a CURRENT measurement, re-derived on the engine as it stands after the last release of this version.**

THE SUN FIX BELONGS ON THIS PAGE FOR ONE REASON, AND IT IS A LESSON ABOUT WHAT A WEATHER OBJECT IS. This document treats the sky as a damage multiplier, because that is the only part of it the damage chain touches. The authority's sunnyday condition also carries an `onImmunity` handler that refuses `frz` , and this engine had read only the condition's damage handlers - so a body under sun could be frozen here and not in the real game. **A weather is not a multiplier; it is an object with handlers, and reading the ones this page cares about is not reading the object.** The refusal is read through effective weather rather than off the field, so an ability that suppresses weather puts the freeze back in both engines - asserted as a control.

WHICH ARTIFACT HOLDS WHICH WHOLE-GAME FIGURE, AND THE TWO-NUMBER PROBLEM IS BACK BY DESIGN. `data/game-differential.json` was NOT republished this pass and still holds board-material 46 of 961; the measurement on tonight's engine is 37 of 961 with protocol 122, and it lives in `data/verification/fix-batch-7.json`. The published figure is stale as a description of the engine and is superseded by the verification artifact until a settled-tree pass rewrites it. Name the artifact whenever either figure is quoted. Neither number is evidence about the damage chain: a clean stage table is evidence about the formula and about nothing else.

5.253.0 - NO DAMAGE STAGE MOVED, NO MULTIPLIER CHANGED, AND NO DAMAGE VALUE WAS RE-MEASURED. THIS RELEASE IS A PARSE REPAIR IN THE DEFECT REGISTER. Nothing in this document's stage table, its multiplier classes or its ordering changes. No mechanic was altered in the simulator, no differential ran and no game was played. The stage-by-stage instrument was NOT re-run in this pass; its result is carried forward from the previous version, where it was measured on the current engine.

WHY A REGISTER REPAIR APPEARS ON A DAMAGE PAGE AT ALL. A row whose status cell begins `open — engine DEFECT` was reporting CLOSED to the gate, because the status reader takes the text after the LAST pipe in the row and a code span inside the cell carried a pipe. Any row on this page's subject could have been hidden the same way, in either direction, and eighteen rows were repaired across the register. **None of the eighteen is a damage-stage row, and no stage, multiplier or roll index is touched by any of them.** The published whole-game figure in `data/game-differential.json` is unchanged at board-material 46 of 961.

5.252.0 - NO DAMAGE STAGE MOVED AND NO MULTIPLIER CHANGED, AND THE CURRENCY THIS PAGE HAS OWED SINCE 5.246.0 IS DISCHARGED: THE STAGE-BY-STAGE INSTRUMENT WAS RE-RUN ON THE CURRENT ENGINE. `tests/test-engine-diff.js` ran on release `0dec37ff5ad9` and rewrote `data/engine-diff.json`, which now carries the release id, its cut time, the Showdown commit and a full set of source digests. The measured result is unmoved: 6,000 comparisons requested, 6,000 compared, 6,000 agreed, **0 disagreed**, at the midpoint and at all 16 corner arms - 17 damage indices, seed 20260804. The scope caveat is unchanged and is stated rather than dropped: `skipped_multihit` 134, `skipped_ability_multihit` 17, 0 non-finite. **So the stage table below is a CURRENT measurement again, not dated evidence**, and the six versions of "the currency is still owed" above it are answered by the instrument rather than by an argument.

WHICH ARTIFACT HOLDS WHICH WHOLE-GAME FIGURE - AND THE TWO-NUMBER PROBLEM IS OVER. `data/game-differential.json` was republished this pass and holds board-material 46 of 961 with protocol first-divergence 141. There is no longer a measured figure sitting outside the published one: the sentence this page has carried for five versions, that the gate clause still reads 77 of 961 while the current measurement sits in a `data/verification/` artifact, is **DISCHARGED**. Name the artifact whenever either figure is quoted, exactly as before; what has changed is that there is now only one artifact to name.

WHY A CLEAN STAGE TABLE STILL DOES NOT CLEAR THE GATE, RESTATED BECAUSE THIS PASS MAKES IT EASY TO MISREAD. The damage chain agrees with the authority at every index and the whole-game clause is RED at 46 games. Both are true and they are not in tension: this table is a per-hit comparison of one calculation, and the whole-game figure is a comparison of boards after a sequence of turns in which address, order, stream and residual all get a vote. A defect of ADDRESS or of STREAM is invisible here by construction, which is what the previous two versions found on the confusion road. A clean 0 of 6,000 is evidence about the formula and about nothing else.

5.251.0 - NO DAMAGE STAGE MOVED, NO STAGE'S MULTIPLIER CHANGED, AND NO DAMAGE VALUE WAS RE-MEASURED. THIS RELEASE IS THE DEFECT REGISTER AND THE COUNTERS. Nothing in this document's stage table, its multiplier classes or its ordering changes. No mechanic was altered in `engine/medicham2-browser.js`. The only edits recorded under this version inside the simulator are counter DECLARATIONS - fields added to the object they were already being incremented on - which change no base power, no multiplier, no roll index and no stage order. The final 85-to-100 per-cent band and its sixteen indices are untouched.

WHY A DECLARATION-ONLY EDIT STILL BELONGS ON THIS PAGE. Four counters were reading `NaN` because they were incremented into an object that never declared them, and one of them went out as `null` in two published artifacts. A counter is how a wire proves it fired; a `NaN` counter is a wire whose proof was fictional. None of the four sits inside the damage chain this table walks, so no figure here is affected - but the general point is the same discipline this document applies to a multiplier, applied to the instrument instead: a value that cannot say where it came from is not evidence, whether it is a stage constant or a fired-counter total.

THE CURRENCY OF THE STAGE-BY-STAGE DIFFERENTIAL IS STILL OWED, ON THE SAME TERMS AS THE PREVIOUS TWO VERSIONS. `tests/test-engine-diff.js` was NOT re-run in this pass. The stage table below is dated evidence from the release it names and is not a current measurement; it is superseded by the next run of that instrument, not by this paragraph. The offered reason at 5.243.0 and 5.244.0 - that nothing feeding it had moved - is not claimed here either, because the simulator's bytes did move even though its behaviour did not, and *the behaviour did not change* is an argument rather than a measurement.

WHICH ARTIFACT HOLDS WHICH WHOLE-GAME FIGURE - UNCHANGED, AND NOT RE-DERIVED BY THIS PASS. `data/verification/fix-batch-M6instr-defog.json` still holds board-material 46 of 961 and protocol 141. `data/game-differential.json` is the artifact the gate clause reads; it was not rewritten and still holds 77 of 961. Name the artifact whenever either is quoted. Neither number moved in this version and neither may be read as evidence that it did.

5.250.0 - NO DAMAGE STAGE MOVED AND NO STAGE'S MULTIPLIER CHANGED. ONE OF THE TWO FIXES IS A DAMAGE VALUE AND IT IS STILL NOT A STAGE: IT IS WHICH RANDOM STREAM THE ROLL COMES OUT OF. Nothing in this document's stage table, its multiplier classes or its ordering changes. The confusion self-hit is not a move and does not walk this chain: the authority computes it inside `getConfusionDamage` and calls the random multiplier DIRECTLY, never entering the wrapped damage function this table is derived against. The fix wraps that method in the instrument and moves the simulator's draw onto the same stream. The final 85-to-100 per-cent band, its sixteen indices and every multiplier above it are untouched, on both roads. The second fix, Defog, is a field sweep and touches no damage stage at all.

THE ONE THING A READER OF THIS TABLE SHOULD TAKE FROM THE PASS. A shared die read in opposite directions is indistinguishable, at the stage table, from two independent dice - and it is worse than independent, because it is anti-correlated. That is a defect of ADDRESS and of STREAM, not of arithmetic, and no stage-by-stage comparison of multipliers can see it. This is the second time in two passes that the confusion road has produced a damage disagreement with every multiplier in this document correct.

WHICH ARTIFACT HOLDS WHICH WHOLE-GAME FIGURE. `data/verification/fix-batch-M6instr-defog.json` holds board-material 46 of 961 and protocol 141. `data/game-differential.json` is the artifact the gate clause reads; it was not rewritten and still holds 77 of 961. Name the artifact whenever either is quoted. The session's full sequence is 77, then 61, then 50, then 53, then 46, and the step that rose is left in.

THE CURRENCY OF THE STAGE-BY-STAGE DIFFERENTIAL IS STILL OWED, AND IT IS STILL WORSE THAN OWED. `tests/test-engine-diff.js` was NOT re-run in this pass, and the simulator moved again, so the reason offered at 5.243.0 and 5.244.0 - nothing that feeds it moved - does not apply here either. The stage table below is dated evidence from the release it names and is not a current measurement. It is superseded by the next run of that instrument, not by this paragraph.

5.249.0 - NO DAMAGE STAGE MOVED AND NO STAGE'S MULTIPLIER CHANGED. THE ONE FIX IS AN ADDRESS AND NOT A MULTIPLIER, AND IT SITS BESIDE THE DAMAGE CHAIN RATHER THAN INSIDE IT. Nothing in this document's stage table, its multiplier classes or its ordering changes. The correction is to WHERE the confusion self-hit takes its two draws from, not to what either draw is worth: `data/conditions.ts` writes `this.activeTarget = pokemon` between the roll for the self-hit and `getConfusionDamage`, so the authority's two draws sit at two different addresses when the confused body has clicked at a foe, and this engine used one address for both. The confusion self-hit does not run through the stage chain this document tabulates - it is `getConfusionDamage`, a fixed 40-power typeless hit - so no row here is affected. **THE ONE THING A READER OF THIS PAGE SHOULD CARRY AWAY IS THAT THE INSTRUMENT HALF IS STILL OPEN AND IT IS A DAMAGE-ROLL ADDRESS QUESTION.** With the addresses matched, the two engines now draw the same value and read it in OPPOSITE directions: the comparator returns the raw index where this engine takes the mirrored one. That cannot be flipped from the engine side without the bottom-tie corner parting on every self-hit, and it is worth all 14 games. Until it is done, the per-hit stage differential and this whole-game figure are answering different questions about the same draw.

THE CURRENCY OF THE STAGE-BY-STAGE DIFFERENTIAL IS STILL OWED, AND IT IS STILL WORSE THAN OWED. `tests/test-engine-diff.js` was NOT re-run in this pass, and the simulator moved again, so the reason offered at 5.243.0 and 5.244.0 - nothing that feeds it moved - does not apply here either. The pinned artifact stranded by the previous release cut has not been regenerated by this pass. Whether `data/engine-diff.json` still describes this engine is unknown rather than merely un-re-checked, and no figure on this page may be quoted as current on the strength of the table alone.

WHICH ARTIFACT HOLDS WHICH WHOLE-GAME FIGURE. `data/verification/fix-batch-M6-sidesel.json` holds board-material 53 of 961 and protocol 154. `data/game-differential.json` is the artifact the gate clause reads; it was not rewritten and still holds 77 of 961. Name the artifact whenever either is quoted.

5.248.0 - NO DAMAGE STAGE MOVED AND NO STAGE'S MULTIPLIER CHANGED. THE ENGINE BYTES UNDER THIS TABLE MOVED AGAIN, AND NONE OF THE THREE FIXES IS INSIDE THE DAMAGE CHAIN. Nothing in this document's stage table, its multiplier classes or its ordering changes. The three corrections in `engine/medicham2-browser.js` are a refusal evaluated against the wrong aim (Sucker Punch into a redirector), a `stall` die addressed as two independent coins rather than one shared one, and a contact ability transfer announcing on the wrong body. The first two decide WHETHER a move resolves at all and the third decides WHOSE boost is announced; none of the three changes a damage multiplier, a roll index or a stage order. The board-material whole-game figure fell 61 of 961 to 50 of 961 and protocol first-divergence fell 161 to 150 across that change, on identical pins with the engine release `f3504e5f88d6` to `9b449a41c865`.

THE CURRENCY OF THE STAGE-BY-STAGE DIFFERENTIAL IS OWED, NOT ASSERTED, AND IT IS NOW WORSE THAN OWED. `tests/test-engine-diff.js` was NOT re-run in this pass and the simulator moved again, so the reason offered at 5.243.0 and 5.244.0 — *nothing that feeds it moved* — does not apply. Cutting the new release additionally STRANDED the pinned `engine-diff` artifact: it names a release the tree has moved past, so its clause cannot speak until it is regenerated. That is the pin guard working rather than a regression, and it means the per-hit result is a figure to WITHHOLD and re-measure. `node engine/provenance.js` decides that question; this document does not restate the per-hit number.

WHICH ARTIFACT HOLDS WHICH WHOLE-GAME FIGURE. `data/verification/fix-batch-M5M7M8.json` holds board-material 50 of 961. `data/game-differential.json` is the artifact the gate clause reads; it was not rewritten and still holds 77 of 961. Name the artifact whenever either is quoted.

5.247.0 - NO DAMAGE STAGE MOVED AND NO STAGE'S MULTIPLIER CHANGED, BUT THE ENGINE BYTES UNDER THIS TABLE DID MOVE, AND ONE OF THE THREE FIXES SITS ADJACENT TO THE DAMAGE PATH RATHER THAN INSIDE IT. Nothing in this document's stage table, its multiplier classes or its ordering changes. What changed is `engine/medicham2-browser.js`, in three places, taking the board-material whole-game figure 77 of 961 to 61 of 961 on identical pins (release `8ad06030e129` → `f3504e5f88d6`). **THE ADJACENCY IS WORTH STATING PRECISELY, BECAUSE IT IS EASY TO OVERSTATE IN EITHER DIRECTION.** The `multiaccuracy` fix changed how many ARRIVALS a volley lands - the authority rolls accuracy per arrival and stops at the first miss, and Triple Axel and Population Bomb are the two carriers in this regulation, derived rather than recalled. It did not change what a single arrival is priced at. Every stage below prices ONE hit, so no stage moved; a volley's total is that price times the number of arrivals, so the whole-game consequence follows from a COUNT and not from a formula. The other two fixes - a non-permanent forme not reverting on switch-out (`clearVolatile` ends with `setSpecies(baseSpecies)`, `sim/pokemon.ts:1564`) and a `choicelock` never cleared - touch no damage stage at all, though a forme carries base stats, so a forme left standing prices later hits off the wrong body. **AND THE MULTI-HIT FAMILY IS THE PER-HIT DIFFERENTIAL'S OWN DECLARED BLIND SPOT**, which is the honest reading of why a volley defect of this size was found by a whole-game comparison rather than by the stage table: the per-hit comparison skips the multi-hit tag by construction and prints that skip on every run.

THE CURRENCY OF THE STAGE-BY-STAGE DIFFERENTIAL IS OWED, NOT ASSERTED. `tests/test-engine-diff.js` was NOT re-run in this pass, and the reason given at 5.243.0 and 5.244.0 - *nothing that feeds it moved* - is not available this time, because the simulator itself moved. Whether `data/engine-diff.json` still describes the engine that exists is a question for `node engine/provenance.js`, which compares content rather than mtimes. This document does not restate a per-hit result measured on engine bytes that have since changed.

WHICH ARTIFACT HOLDS WHICH WHOLE-GAME FIGURE. `data/verification/fix-batch-M1M3M4.json` holds board-material 61 of 961. `data/game-differential.json` is the artifact the gate clause reads; it was not rewritten in this pass and still holds 77 of 961. Both are correctly measured on identical pins, and only the second is published.

5.246.0 - NO DAMAGE STAGE MOVED AND NO DAMAGE NUMBER MOVED. THE SECOND-ORDER NOTE PUBLISHED IN 5.245.0 IS RETRACTED, BECAUSE THE INSTRUMENT BEHIND IT WAS WRONG. Nothing in this document's stage table, its multiplier classes or its ordering changes. What changes is a note attached beneath it. The 5.245.0 block below states that *"every body in this engine that loses an item gets Unburden's speed doubling"* and reasons from it about turn order. **That sentence is withdrawn**; the block stands as published and is superseded from here rather than rewritten. `engine/medicham2-browser.js:14770` is the ENTRY GUARD only, and the multiplier on the next line, `:14772`, is gated on `TAGS.param('ability', m.ability, 'speedOnItemLoss')` — a parameter the ability block of `data/tags.json` gives to exactly one key, `unburden`. A census control already on record reads `ability none 187,187` for the arm that must not move against `Unburden 187,374` for the arm that must. **IT SURVIVED LONG ENOUGH TO BE PUBLISHED BECAUSE OF AN INSTRUMENT DEFECT, NOT AN ENGINE ONE.** `tests/probe_leaf_widening.js:277` compared its own stand-in for *this body has lost an item* against the authority's `unburden` volatile, and never read this engine's Speed; an agreement therefore said that both bodies had lost an item and nothing else. **An instrument can be wrong before the engine is** — the discipline this document applies to a damage stage, applied to the ruler instead of to the game. **WHAT REMAINS IS NARROWER AND STILL TOUCHES NO STAGE HERE.** ROADMAP #535: the doubling is recomputed from the body's CURRENT ability rather than held from the moment the item was lost, so an Unburden acquired afterwards doubles here and not

in the authority, with Skill Swap the reachable door. Filed `INSTRUMENT OWED` — nothing decides it yet. Speed still enters no stage of this document's chain; it decides TURN ORDER, so a damage figure that moves because of it has had no damage stage change under it. `tests/test-engine-diff.js` was not re-run in this pass and no damage figure in this document is re-derived by it.

5.245.0 - NO DAMAGE STAGE MOVED AND NO DAMAGE NUMBER MOVED, AND FOR THE FIRST TIME `data/engine-diff.json` **CAN PROVE WHICH ENGINE PRODUCED THE ZERO.** Nothing in this document's stage table, its multiplier classes or its ordering changes. What changed is the artifact's PROVENANCE: `engine-diff` was regenerated and now carries 26 `source_digests` plus a `showdown_commit`, read back field by field rather than trusted, behind one door (`engine/pin_guard.js`) that withholds a figure when the pin is absent, wrong or unverifiable. **The measured figure is unmoved: 0 of 6,000 at the midpoint and at both corners.** That regeneration was impossible for most of the day: a guard added hours earlier ran at MODULE LOAD off the whole process's argv and killed `tests/roster.js --write` and `engine/all_mechanics_fire.js --write` at exit 2 — the SKIP code — so the runner would have reported them politely skipped rather than failed; it now tests `require.main === module`. **THE WHOLE-GAME EVIDENCE THAT SITS BESIDE THIS TABLE IS NOW A DIFFERENT QUANTITY, AND THAT IS A REVERSAL.** The gating whole-game clause counted protocol first-divergence and printed **167 of 961**; it now counts BOARD-MATERIAL games, `state.games` less `state.games_board_never_diverged` = 961 − 884 = **77 of 961 (8.0%)**, with narration carried by its own non-gating clause at 167. Any sentence in the dated blocks below that reads 167 as *the* whole-game number is superseded here rather than rewritten, and neither figure may be stated without naming which quantity it is. The gate reads `CLOSED — 1 of 8 GATING clauses fail`. **ONE SECOND-ORDER NOTE OF THE KIND THAT LOOKS LIKE A DAMAGE CHANGE LATER, AND IT IS NOT ONE.** `engine/medicham2-browser.js` holds no Unburden state: `effSpeed` recomputes the doubling from `_hadItem && !m.item` (`:14770`), so **every body in this engine that loses an item gets Unburden's speed doubling** — a Knock Off, a consumed berry, a spent Focus Sash. Speed enters no stage of this document's chain. It decides TURN ORDER, so it changes which hit is dealt from what remaining HP and which body is still standing to be hit at all; a damage figure that moves because of it has not had a damage stage change under it. Filed as an ENGINE defect owed a register row, not fixed here. `tests/test-engine-diff.js` was not re-run beyond the regeneration above, because nothing that feeds the damage chain moved: the only engine-adjacent edit this version is `engine/board_state.js`, the COMPARATOR, which is not one of the 26 frozen sources and left the release id at `8ad06030e129`.

5.244.0 - NO DAMAGE STAGE MOVED AND NO DAMAGE NUMBER MOVED. THE RULER OVER THE DAMAGE TABLE STOPPED LISTING THE FIELDS IT HASHES AND NOW DERIVES THEM, AND THE DERIVATION FOUND A SECOND BLIND CLASS THE LIST COULD NOT HAVE FOUND. Nothing in this document's stage table, its multiplier classes or its ordering changes, and `tests/test-engine-diff.js` was not re-run because nothing that feeds it moved: no engine byte changed in this pass. The change is again to a RULER rather than to the formula, and it is the SECOND pass over the same ruler, which is the part worth recording here. **WHY 5.242.0 WAS NOT THE FIX, STATED AS A REVERSAL.** That version repaired two named terms in `engine/feature_fixture.js` 's `tableDigest()` — typing, which was hashed under a field name no row carries, and weight, which was not hashed at all — and this page recorded it as done. It was INSTANCE-level: the hashed set was still an ENUMERATED LIST, so a field added to the damage table tomorrow goes unhashed exactly as typing did, and the class of failure was re-armed the moment the two instances were closed. The hashed set is now DERIVED from the rows' own keys, minus a declared exclusion map of six entries. **THE SECOND BLIND CLASS, WHICH ONLY THE DERIVATION COULD SURFACE.** An ABSENT field hashes identically to a field that is PRESENT AND EMPTY. So deleting a body's entire moveset moved the digest not at all — live on **4 rows for** `mv` **and 10 for** `item`. Both reach this document's chain: `mv` decides what a body can throw and therefore which base power, category and type ever enter the formula, and `item` sits in the multiplier walk. A table change on either could have passed under a fitted vector with the alarm silent. **THE DIGEST VALUE IS UNCHANGED AT** `9d289cf77e24` **.** This adds no third reason to restamp. The two reasons recorded at 5.242.0 — the damage table was regenerated AND the ruler that measures it changed — are still the two, they still cannot be separated by a single restamp afterwards, and the verdict remains RESTAMP-not-refit and the owner's call. **ONE MORE THING THAT COULD NOT SEE THIS TABLE.** `tests/test-artifact-keys.js` , the check written to catch a hand-typed lookup, sliced to 8 keys, capped depth at 3 where the deepest real object is 10, never descended arrays and exited 0 when the engine data failed to load — so `MC.priors` , **230 keys**, was never inspected once. No damage number moves because of the repair; what moves is that a wrong lookup in that table is now visible. **A SILENT RULER AND A RULER THAT CANNOT SEE PRINT THE SAME OUTPUT**, which is why both of these were found by mutation rather than by reading, and why the mutation pilot this version records — **83 mutants, 57 killed, 26 survived** — was aimed at the gates and not at the engine: the damage path has the official simulator as an oracle and the rulers over it have none. **5.243.0 - NO DAMAGE STAGE MOVED AND NO DAMAGE NUMBER MOVED, BUT THE WHOLE-GAME EVIDENCE THAT SAT BESIDE THIS TABLE WAS MEASURED WHERE THE GAMES DO NOT END.** Nothing in this document's stage table, its multiplier classes or its ordering changes, and `tests/test-engine-diff.js` was not re-run because nothing that feeds it moved: the stage-by-stage damage differential is a per-hit comparison and is not steered by a driver at all. What changes is the standing of the OTHER instrument this document has leaned on for whole-game confidence. The whole-game clause of the gate was answered by an artifact from the `census-coverage-seeking/v1` driver, which seeks census coverage rather than winning: 17 of 961 games reached a result (1.8%), 944 stopped at the 12-turn cap with both sides standing, and 0 boards parted. On identical pins — release `8ad06030e129` , cap 12, pool `0d103fb9fa87` , census pin `9446a684709d` , 961 games of a 1200 PAIR budget, the driver the only difference — the `empirical-click/v1` driver reaches a result in 474 of 961 (49.3%) and 77 boards part. The corrected whole-game figures are therefore 77 of 961 (8.0%) board-material and 168 protocol first-divergence, and the gate reads CLOSED with 1 of 8 clauses failing. Any sentence anywhere that treated a clean whole-game differential as corroboration for this damage path is superseded, including in this document's own dated blocks below, which are kept as written. **WHY THAT MATTERS ON THIS PAGE SPECIFICALLY.** A damage error that only shows itself after a body faints, after a forced switch, or in the last third of a game is invisible in a population where 944 of 961 games stop at the turn cap with both sides standing. The per-band damage agreement recorded in this document is a DIRECT hit-for-hit comparison and stands on its own evidence; the whole-game reading is a different instrument and no longer corroborates it. Nothing was tuned. **5.242.0 - NO STAGE MOVED AND NO DAMAGE NUMBER MOVED; THE ALARM ON ONE DAMAGE INPUT HAD NEVER BEEN ARMED.** Nothing in this document's stage table or multiplier classes changes. The change is to a RULER rather than to the formula: `engine/feature_fixture.js` 's `tableDigest()` decides whether the damage table underneath a fitted vector has moved, and it was blind to two of that table's fields. It hashed `m.ty` - present on 0 of 322 rows, because this table spells typing `t` - so the term was a constant `null` and a TYPE change never reached it. It did not hash `wt` at all. **WEIGHT BELONGS IN THIS DOCUMENT BECAUSE WEIGHT IS AN INPUT TO THE DAMAGE NUMBER, NOT A LABEL BESIDE IT.** Six entities in this format read weight or change it - Low Kick, Grass Knot, Heavy Slam, Heat Crash, Heavy Metal and Light Metal - so a species weight edited in the table reaches the damage number at §1 stage 1, the BasePower chain. Typing is the same exposure ten rows further down the same pipeline: §1 stage 11, type effectiveness, reads what `t` says. Both fields are hashed now. **THE BLINDNESS WAS SHOWN BY MUTATION, NOT BY READING.** Mutating one row's `t` did not move the digest, nor did `wt` ; `mv` and `st` did, which is the control that proves the harness can see a change. After the fix all four move. Inventing a `ty` field moved the digest BEFORE the fix, proving the term was live in the hash and the data had never populated it - an absent field and a present-but-empty field hash identically, so nothing short of a mutation could have told them apart. **NOTHING IN THE DAMAGE PATH MOVED.** The digest moved `1bda9df11d73` -> `9d289cf77e24` **with no change to the table itself**, so no stage, no multiplier and no damage number in this document is affected in either direction, and no damage measurement was re-run or needed re-running. **NO RESTAMP AND NO REFIT WERE RUN, BY THE OWNER'S EXPLICIT DECISION** - MAG stays paused until MEDICHAM is correct, and that is a decision rather than an omission. Full account: `docs/_reports/2026-09-03-table-digest-blind-fields.md` . **5.241.0 - NO STAGE MOVED, NO DAMAGE NUMBER MOVED ON THE DIRECT ROAD, AND ONE ROAD GAINED A STAGE IT NEVER HAD.** The delayed payout

(`data/conditions.ts:415` -> `trySpreadMoveHit`) takes the authority's FULL step list, so the crit step at `sim/battle-actions.ts:1156` -> `:1636-1642` applies to it exactly as to a direct click, with `critMult = [0,24,8,2,1]` at `:1633` , the x1.5 at `data/mods/champions/scripts.ts:222` and the `|-crit|` line at `:285` . This engine drew no crit for that road at all - not rarely, never - so the stage was ABSENT rather than misplaced. It is now present, and it is present at the CORRECT position: the payout is re-priced through the certain-crit path (§3, the road that already passes at both endpoints), not multiplied onto a number that has already been rolled, STAB'd, type-charted and chain-spent, which is the battle loop's rolled-crit defect §3 still records. **The evidence is the unwired-knob signature, and it is the reason a stage can be absent for months without a divergence naming it:** handed a crit-certain die and then a crit-impossible one, the authority answered 72 with the line and 48 without while this engine answered 69 both times. **Nothing in §1's stage table changes**, and no multiplier moved class in §2. **The second fix is downstream of the formula, not in it.** A hit that overkills a Substitute doll now books what the DOLL absorbed rather than what the BODY could have taken (`data/moves.ts:18341-18357` , no Champions override): the damage the formula produced is unchanged, and what changed is the quantity recoil and drain are paid FROM - recoil -25 became -12, drain +62 became +21, both now equal to the authority. **THE CORNER DAMAGE DIFFERENTIAL WAS NOT RE-RUN IN THIS PASS** - the machine was in light mode - so the previous release's figure must NOT be read as covering this change. A road that gained a crit stage can move a damage number, which is exactly what that instrument exists to see; the command is in the report's OWED block and the figure is withheld until it runs. **5.240.0 - NO DAMAGE STAGE MOVED AND NO DAMAGE NUMBER MOVED.** This version changes whether Electric Terrain lets a sleep or a drowse land. Nothing in the damage chain is touched, and `tests/test-engine-diff.js` was re-run anyway on the same seed as a control: 6,000 compared, 6,000 agreed, 0 disagreed. One second-order note of the kind that looks like a damage change later: a body that stays awake keeps acting, so games under Electric Terrain can now run differently from that point - that is the board changing, not the formula. **5.239.0 - NO DAMAGE STAGE MOVED AND NO DAMAGE NUMBER MOVED.** This version changes where a hazard punish puts its layer and when a weather punish is allowed to set its sky. Neither touches the damage chain, so `tests/test-engine-diff.js` was not re-run and its `0 of 6,000 at all sixteen corners` stands unchanged. One second-order note, recorded because it is the kind of thing that looks like a damage change later: Sand Spit now sets sandstorm over a standing sun or rain, so the sun/rain Fire and Water multipliers stop applying from that point in those games - that is the weather changing, not the formula. **5.237.0 - NO DAMAGE STAGE MOVED AND NO DAMAGE NUMBER MOVED.** This version changes the accuracy chain, which decides WHETHER a move connects and sits above every damage step; `tests/test-engine-diff.js` was therefore not re-run and its `0 of 6,000 at all sixteen corners` stands unchanged. Recorded here because the accuracy stage table `(3+n)/3` is a near neighbour of the stat-stage table `(2+n)/2` and the two are easy to confuse - they are separate constants and only the accuracy one was touched.

5.236.0 - NO STAGE MOVED AND NO DAMAGE NUMBER MOVED. This version changes how many times the King's Rock flinch die is drawn on a multi-hit move (once per landed arrival, not once per click). The draw is a SECONDARY, taken after the damage is already applied, so no stage in this document is touched and `tests/test-engine-diff.js` was deliberately not re-run - its standing result is 0/6000 at all sixteen corners, seed 20260804, 2026-08-29, with the unchanged `skipped_multihit 134` scope caveat.

5.235.0 - NO STAGE MOVED, NO DAMAGE NUMBER MOVED, AND NOTHING IN THIS DOCUMENT CHANGES. This version adds the collector for the next regulation - a detector that derives the target Showdown format at run time instead of holding a constant, and one store per format id - and repairs the per-game format tag in `engine/durable-ingest.js` , which returned a literal for every Champions tier. Neither touches the damage path: no multiplier, no stage, no rounding rule and no order of operations was read or edited, and no figure here was re-measured. It is recorded only so the version line does not assert a damage change it did not make. Full account: `docs/_reports/2026-08-31-next-regulation-ingest.md` .

5.234.0 - NO STAGE MOVED, NO DAMAGE NUMBER MOVED, AND TWO DAMAGE DIVERGENCES BECAME VISIBLE THAT HAD BEEN LABELLED IMPOSSIBLE. Nothing in this document changes. The divergence annotator - the instrument that tags each disagreement with whether the entities it names are legal in this format - was resolving a token to the first dex hit, and `|-damage|p1a:floette|74/149` against `92/149` names Floette-Mega / Floette-Eternal, both legal, through a base spelling that is `Past` and `tier: 'Illegal'` . Both such rows are `board_parted: 1` , `DIFFERENT-END-STATE` , and both had been binned `cannot_occur_in_format: true` . **They are** `-damage field 3` **MAGNITUDE divergences and nothing here yet says which stage they belong to** - no stage was investigated, no multiplier was touched, and no figure in this document was re-measured. They are recorded here so that the next damage-stage pass has them on the list rather than filtered out of it. Full account: `docs/_reports/2026-08-31-annotator-entity-kind.md` .

5.233.0 - NO STAGE MOVED, AND A BASE POWER DID - WHICH IS THE STAGE BEFORE THE STAGES. Low Kick, Grass Knot, Heavy Slam and Heat Crash carry no printed base power and get their real one from a `basePowerCallback` keyed on weight; the Champions mod overrides none of the four, so all four inherit mainline verbatim, and the kg-converted bracket table the engine reads agrees with the hectogram table in the authority. Ten rows of the mon table carried no `wt` , so `effWeight` returned null and all four fell through to the dex base power on a body BUILT at one of them, and to the weight of the body that LEFT the field on a forme change. The artifact is regenerated and the generator's own row census now reports no weightless row. **The multiplier chain is untouched and no damage FORMULA changed**; what changed is the number that enters it as base power. Measured on a real turn against a fixed-base-power move of the same type and category, so every multiplier the forme change moved cancels in the ratio (`data/mechanics-census.json` , `move/variablePower`): Skarmory 50.5->40.4 kg steps the Low Kick / Brick Break ratio DOWNWARD by 25.8%, the only such crossing in this format; Victreebel 15.5->125.5 kg steps 142.4%; and Falinks 62->99 kg crosses no bracket and steps 2.8% identically before and after, which is the cleared control. On the built-at door the three Gourgeist sizes at 9.5, 14 and 39 kg land on 2.00x and 3.00x of the smallest, against a Gourgeist that already carried its weight at 12.5 kg and 1.00x. **The pool's one moved damage value is the RATIO family and not the bracket table** - a Heat Crash at a freshly mega-evolved Falinks, whose user-over-target ratio crosses a bracket even though its target weight does not. The arithmetic and the protocol lines are in `docs/_reports/2026-08-30-engine-data-regen.md` .

5.232.0 - NO STAGE MOVED AND NO DAMAGE NUMBER MOVED; HP MOVED, AND IT IS A HEAL ON THE ATTACKER RATHER THAN A STAGE. Bug Bite and Pluck now make the thief eat the berry they took, so a stolen Sitrus pays `maxhp/4` to the ATTACKER through `berryForceEat` - the same shared `singleEvent('Eat')` implementation Stuff Cheeks, Teatime and Cud Chew already use - and the 18 resist berries pay nothing, because their `onEat` body is empty in the authority too. Nothing in `dmgRange` or in the multiplier chain was touched: the resist berry's own halve is applied where it always was, and the probe's Chople arm asserts the thief's HP does NOT move on a berry with an empty effect, which is what separates the heal from a stage. **The one damage-shaped thing still owed here is Ripen's SECOND halve**, which is a `chainModify(0.5)` at `onSourceModifyDamagePriority: -1` gated on `abilityState.berryWeaken` . Measured at 94 against a required ~47 on an Ice Beam into an Appletun holding a Yache, with `MEDFAILS.damageReduceUnknown` naming `ripen/null` - the reader refuses and says so rather than defaulting on. It is blocked on a `tag_dex.js` regeneration, not on the diagnosis.

5.231.0 - NO STAGE MOVED AND NO DAMAGE NUMBER MOVED; HP MOVED, AND THAT IS A HEAL RATHER THAN A STAGE. The type-resist berry's halve is applied by `dmgRange` as a pure read and always was - the probe's empty-hand arm asserts it exactly, taking 296 against the berry arm's 148 both before and after this change - so what moved at that site is only WHO gets told the berry was eaten. `consumeBerry` now runs there, which raises `runEvent('EatItem')` , and a Cheek Pouch holder therefore takes a `maxhp/3` heal between the `-enditem [eat]` and the `-enditem [weaken]` , exactly where `chopleberry.onSourceModifyDamage` 's `if (target.eatItem()) { ... }` puts it. That heal lands during the damage CALCULATION and before `spreadDamage` moves any HP, which is the authority's own position and not a choice. **One thing worth stating for the next reader of this file**: Ripen's

`onSourceModifyDamage` is a SECOND `chainModify(0.5)` gated on `abilityState.berryWeaken`, written from `onEatItem`. This engine's `damageReduce` row for Ripen carries `onlyWhen: null` and the reader REFUSES it rather than defaulting on, counting `MEDFAILS.damageReduceUnknown` - so that multiplier is still absent from this stage, and it is absent DELIBERATELY. Raising the event is its prerequisite; wiring the multiplier is filed as its own batch. `tests/test-engine-diff.js` was NOT re-run: it calls `moveHit` once, has no `Update` pass and no residual berry, so it is structurally blind to every road this batch touched - an argument, not a measurement, recorded as one.

5.230.0 - NO STAGE MOVED, AND ONE DAMAGE INPUT DID — DECLARED RATHER THAN LEFT TO BE FOUND. Both changes decide WHERE a line is written: the busted-disguise `detailschange` and its `maxhp/8` chip move from the hit to the `Update` at the foot of the hit, and Cheek Pouch's heal moves below the berry's own. Neither touches the packet vector, the roll index or the crit decisions, all of which are drawn in `_stepDamage` above all of this. **The declared change: on a single-arrival click into an intact Disguise, `dmg` now reads 0 rather than the chip.** That is `onDamage`'s own return (`data/abilities.ts:962-967` returns 0, so `damage[i]` is 0) and the chip is a separate `this.damage(baseMaxhp/8, pokemon, pokemon, species)` whose source effect is a SPECIES and not a Move — so no Focus Sash, no Endure and no recoil may answer it, and those three blocks sit below the absorb and read `dmg`. It is strictly closer to the authority than the value that used to sit there, it is reachable only on a body an intact Disguise is absorbing for, and it is stated here because it is a state change riding in an ordering batch. **The HP is unchanged and the probe asserts it**: the holder ends on `maxhp - maxhp/8` on the spread arm and on the plain arm, and the Cheek Pouch body ends on the same total with the fix and without it — only what the berry's own `-heal` line REPORTS moved (370/596 now against 568/596 before, the post-both total), which is asserted against a no-ability control. **A pre-existing damage divergence was measured in the same pass and filed rather than fixed**: a type-resist berry never routes through `consumeBerry`, so Cheek Pouch pays nothing for it — Close Combat into a Cheek Pouch Maushold holding a Chople Berry emits `-enditem [eat]`, `-enditem [weaken]`, `-damage` and no `-heal`. `tests/test-engine-diff.js` was NOT re-run: it calls `moveHit` once, has no `Update` pass and no residual berry, so it is structurally blind to both changes — which is an argument and not a measurement, and it is recorded as one.

5.229.0 - NO STAGE MOVED AND NO DAMAGE NUMBER MOVED. The resist berry's halve is applied by `dmgRange` as a pure read and always was; only the CONSUMPTION and its two `-enditem` lines moved, from the apply step up into the calculation step where `getDamage` raises `ModifyDamage`. The arithmetic control is in the probe rather than in this sentence: the same board with an empty hand takes exactly double (79 against 158). The `DamagingHit` change moves WHEN a reaction is paid within a volley, never how much: the toll, the boost and the arrival amounts are unchanged, the packet vector and the roll index are drawn in `_stepDamage` before any of this runs, and the per-arrival crit decisions are untouched. **One reachable path is named rather than left to be found**: an attacker killed by an interior arrival's toll now dies mid-volley, and `hitStepMoveHitLoop` breaks there (`scripts.ts:534-537`) while this packet loop tests only the TARGET's HP - so the volley still runs to its drawn length. Nothing in the 961 pinned games stages it. **A pre-existing damage divergence was measured in the same pass and filed rather than fixed**: a multi-arrival volley halves EVERY arrival against a resist berry where the authority halves only the one that ate it (Triple Axel into a Yache Berry reads 1464 -> 1434 -> 1374 -> 1284 with the berry against 1404 -> 1284 -> 1104 without). `tests/test-engine-diff.js` was NOT re-run: it calls `moveHit` once and cannot see a multi-hit at all, which is an argument and not a measurement, and it is recorded as one.

5.228.0 - NO STAGE MOVED AND NO DAMAGE NUMBER MOVED. Both changes decide WHEN a residual chip or a residual counter runs relative to the others, never HOW MUCH it takes: the burn is still `max(1, floor(maxhp/16))`, ordinary poison still a flat eighth, Toxic still the escalating sixteenth, and the Perish Song step ticks a counter and calls `faint()` without computing damage at all. Every one of them sits in the end-of-turn walk, below `_stepApply` and below the whole hit loop, so the packet vector, the roll index and the crit decisions are untouched. The one path by which an ordering change could reach a damage number is a body dying earlier or later and therefore taking a different number of chips - and the amounts and the totals are unchanged in every staged arm, including a four-body mutual perish wipe that is byte-identical before and after. `tests/test-engine-diff.js` was NOT re-run: it calls `moveHit` once and has no residual walk, so it is structurally blind to this change - which is an argument and not a measurement, and it is recorded as one.

5.227.0 - NO STAGE MOVED AND NO DAMAGE NUMBER MOVED. The change decides WHEN a `boostsOnKO` payment is made, HOW LARGE one payment is, and whether it is made at all once the drain has emptied a side. All three are downstream of every damage step: the step is inserted below `_stepDrainFaints`, which is itself below `_stepApply`, so the packet vector, the roll index and the crit decisions are untouched. The BOOST it grants can reach a later turn's damage, and that is exactly the case the fix removes - a payment made after the battle ended has no later turn to reach. The `single` arm of `tests/probe_afterfaint_boundary.js` is the control that holds this: a single KO on a continuing battle reads `faint,ABIL,BOOST:atk+1` on both engines, before and after. `tests/test-engine-diff.js` was NOT re-run; it calls `moveHit` once and has no faint drain, so it is structurally blind to this change - which is an argument and not a measurement.

5.226.0 - NO STAGE MOVED AND NO DAMAGE NUMBER MOVED. The change decides how many times an `onDamagingHit` REACTOR is raised for a volley that stopped at a KO; it is downstream of every damage step, and the packet vector, the roll index and the crit decisions are untouched. The damage a reactor DEALS is unchanged - `punishesAttacker`'s `fraction` and `dealsDamageTaken` arithmetic is not edited - only the number of times it is charged. The survivor arm of `tests/probe_volley_reactor_count.js` is the control that holds this: identical HP and an identical `-hitcount` on both engines, before and after. `tests/test-engine-diff.js` was NOT re-run; it calls `moveHit` once and skips every `multiHit` move by construction, so it is structurally blind to this change - which is an argument and not a measurement.

5.225.0 - NO STAGE MOVED AND NO DAMAGE NUMBER MOVED. The change decides WHICH SIDE a body that the aim already resolved is looked up on, which is above every damage step; no multiplier is added or removed and no feature read changes. The damaging forced-switch door runs AFTER the hit, so the drag it now performs for an ally-aimed Dragon Tail happens on a board the damage pipeline has already finished with. The two FOE-axis control arms of `tests/probe_ally_forced_switch.js` are line-identical clean and under the knob, which is the same claim measured. `tests/test-engine-diff.js` was NOT re-run - it has no `--out` and would republish the artifact the `0 of 6,000` is read from.

5.224.0 - NO STAGE MOVED AND NO DAMAGE NUMBER MOVED. The change decides WHICH BODY a repeated move is aimed at, not what it does when it lands. No multiplier is added or removed and no feature read changes; the same damage pipeline runs against a different defender. The one adjacent effect is that a spread or `randomNormal` repeat is now PRICED against the recorded slot rather than against the hardest-hit foe — a valuation field on the action, not a stage — and the class gate keeps the loc out of the spread road entirely. The stage-by-stage comparison against the authority is unaffected; `tests/test-engine-diff.js` was re-run and reads `disagreed 0`.

5.223.0 - NO STAGE MOVED AND NO DAMAGE NUMBER MOVED. The change is a REFUSAL: a move that would have queued a second action does not queue it. Nothing in the damage pipeline is touched, no multiplier is added or removed, and no feature read changes. What changes is HOW MANY damage calculations a turn contains — a shielded body no longer takes a swing it was never given. The stage-by-stage comparison against the authority is unaffected and was not re-run.

5.222.0 - NO STAGE MOVED, NO DAMAGE NUMBER MOVED, AND NO FEATURE READ MOVED. This version changes test instruments, one run wrapper and the defect register. It touches no damage packet and no input to any stage in the table below. `data/engine-diff.json` was not rewritten and its modification time is unchanged, so the published **0 of 6,000 at all sixteen roll positions** still stands from 02:49.

CORRECTION TO THE 5.221.0 NOTE BELOW, WHICH IS LEFT STANDING RATHER THAN EDITED. That note gives as its reason for not re-running the differential that the check "has no `--out` and would republish that artifact". That was true when it was written. `--out` **now exists**: it names a path for the run's artifact, refuses any path outside `data/verification/`, and leaves the published artifact alone. So the reason has changed even though the decision has not - the differential still was not re-run for a figure in this pass, and the published residual still stands from 02:49 rather than from today.

For the record, and because it bears on how anyone reads this instrument: the damage residual has never been this check's exit code. `disagreed` is published to the artifact and read by `engine/quarantine.js`. The only three conditions that set a failing status are the accuracy, accuracy-modifier and substitute-bypass conformance sections.

5.221.0 - NO STAGE MOVED, NO DAMAGE NUMBER MOVED, AND NO FEATURE READ MOVED EITHER. The change in this version decides WHETHER THE USER OF A STATUS MOVE LEAVES THE FIELD after that move resolved. It touches no damage packet: `partingshot` has `basePower: 0` and `category: 'Status'`, so no stage in the table below is reached at all on the turn in question, and none of the inputs to any stage changed. Unlike 5.220.0 this batch touches nothing `board.js` reads, so `benchRisk` is unmoved and no refit is owed.

`data/engine-diff.json` was not rewritten and `tests/test-engine-diff.js` was not re-run - it has no `--out` and would republish that artifact. The published **0 of 6,000 at all sixteen roll positions** therefore still stands from 02:49, and it is unchanged by construction rather than by assertion: that instrument compares `dmgRange` outputs and never reaches a `selfSwitch`.

5.220.0 - NO STAGE MOVED AND NO DAMAGE NUMBER MOVED. The change in this version decides WHETHER A MOVE IS REFUSED AT ALL, above every damage step: five priority gates now compare the ability-modified priority instead of the printed constant. A refused move produces no damage packet, so no stage in the table below is reached differently and none of its inputs changed. `data/engine-diff.json` was not rewritten - the published **0 of 6,000 at all sixteen roll positions** still stands from 02:49.

ONE FEATURE READ DID MOVE AND IT IS DECLARED HERE TOO. `clickFragility`'s "blocks priority outright" clause now reads the same number, so `benchRisk` moves for the format's single Gale Wings carrier. That is a FEATURE, not a damage stage - `dmgRange` is untouched - and it is owed a refit at the next release cut.

`tests/test-engine-diff.js` **WAS DELIBERATELY NOT RE-RUN THIS PASS**, because it has no `--out` and would republish that artifact. That instrument compares `dmgRange` outputs directly and never calls a priority gate or `clickFragility`, so it is unchanged by construction; the probe's `nogalewings-bravebird`, `galewings-damaged` and `quickguard-priority0` arms carry HP fractions that are identical between the two engines and on both loads.

5.219.0 - NO STAGE MOVED AND NO DAMAGE NUMBER MOVED. The change in this version decides WHICH BODY a substituted move addresses, above every damage step - a move an Encore forced or a Copycat produced now takes its target from its own target class instead of always from the foes. Nothing above or below the stage table is touched, and `data/engine-diff.json` was not rewritten: the published **0 of 6,000 at all sixteen roll positions** still stands from 02:49.

`tests/test-engine-diff.js` **WAS DELIBERATELY NOT RE-RUN THIS PASS**, because it has no `--out` and would republish that artifact. The probe carries the evidence instead: its `encore-aurasphere` and `copycat-aurasphere` arms are line-identical between the two engines and fail if the far-side draw consumes a different die, which is the only way this change could reach a damage number.

5.218.0 - NO STAGE MOVED AND NO DAMAGE NUMBER MOVED. The change in this version is a SHIELD-GATE one: a shield whose move was substituted mid-turn now passes the queue scan and the consecutive-use roll, where before it passed neither. Nothing above or below the stage table is touched, and `data/engine-diff.json` was not rewritten — the published **0 of 6,000 at all sixteen roll positions** still stands from 02:49.

RE-RUN THIS PASS, and it went to `data/verification/engine-diff.n150.json`: `tests/test-engine-diff.js` reports **0 disagreements over 150 matchups** at seed 20260804. The rc=3 a reader may see is that file's pool advisory rather than a disagreement, A/B verified as pre-existing on release `cc7dca43e395`.

5.217.0 - NO STAGE MOVED AND NO DAMAGE NUMBER MOVED. The change in this version is a TURN-ORDER one: a mid-turn Encore now relocates its target's queued action into the ENCORED move's priority bracket, which is where Champions' own `encore.condition.onStart` puts it. Nothing above or below the stage table is touched - the same move deals the same amount, at a different point in the turn. **RE-RUN THIS PASS, unlike 5.216.0:** `tests/test-engine-diff.js` reports **0 disagreements over 150 matchups**, seed 20260804, and it wrote to `data/verification/engine-diff.n150.json` - `data/engine-diff.json` was NOT touched and the published **0 of 6,000 at all sixteen roll positions** still stands from 02:49. The same run was repeated against `git show HEAD:engine/medicham2-browser.js` swapped into the tree and produced the identical verdict and the identical exit code, so the rc=3 this file's reader may see is the pool advisory for 9 undrawable species and not a disagreement.

5.216.0 - NO STAGE MOVED AND NO DAMAGE NUMBER MOVED. The change in this version is a STATUS REFUSAL: Safeguard now refuses what the target's own partner wrote, which is a `setStatus` and an `addVolatile` question and never reaches the damage chain. `tests/probe_ally_safeguard.js` asserts it explicitly - an ally's Earthquake into the shielded body deals a bit-identical amount on both arms of the Safeguard knob, which is a probe reading and is not a figure from this table. The 6,000-comparison damage differential is expected unchanged at **0 disagreements at all sixteen roll positions**, seed 20260804; it was NOT re-run this pass, deliberately, because `tests/test-engine-diff.js` has no `--out` and would republish `data/engine-diff.json`.

5.215.0 — NO STAGE MOVED AND NO DAMAGE NUMBER MOVED. The change in this version is to a MEASUREMENT INSTRUMENT — the roster's move arm now asks whether a one-turn state ever refused anything — and touches nothing above or below the stage table. The 6,000-comparison damage differential is unchanged at **0 disagreements at all sixteen roll positions**, seed 20260804.

5.214.0 — NO STAGE MOVED, AND NO DAMAGE NUMBER MOVED. The change in this version is a TARGET SELECTION defect, not a damage one: a pivot status move was not offered to the redirection event, so its stat drops landed on the body it named instead of on the redirector. Nothing above or below the stage table was touched. The 6,000-comparison damage differential is unchanged at **0 disagreements at all sixteen roll positions**, seed 20260804, which is the reading this document exists to hold.

5.213.0 — THE GRASS KNOT AND HEAVY SLAM ROWS THE STANZA BELOW CALLED "THE SHARPEST OPEN DAMAGE QUESTION IN THIS DOCUMENT" ARE CLOSED, AND NO STAGE MOVED. Neither was a stage and neither was a roll: base power is computed ABOVE the stage table, and this engine held the body's weight as a build-time constant while the authority rewrites it at every forme change (`Pokemon#setSpecies`, `sim/pokemon.ts:1402`). A mega evolution was therefore priced off the body that left the field — Grass Knot into a Staraptor-Mega read the base forme's 24.9 kg (bracket `>=10`, BP 40) where the authority read 50 kg (bracket `>=50`, BP 80), and Heavy Slam by a Steelix-Mega read a 400/120 ratio (BP 80) where the

authority read 740/120 (BP 120). Predicted damage ratios 0.500 and 0.667 against observed 0.478 and 0.677. **The remaining two weight cards ARE roll residue** (Low Kick 0.947, Heat Crash 1.126) and are correctly not a stage question either. The 6,000-comparison damage differential is unmoved at 0 disagreements at both endpoints, seed 20260804. Full account: `docs/_reports/2026-08-29-weight-base-power.md`.

5.212.0 — NO DAMAGE STAGE MOVED, AND THE POPULATION THE `-damage field 3` CARDS WERE COUNTED IN DID. The whole-game differential's forced-switch mirror was stopping 42 of 961 empirical-arm games on a harness fault rather than on the game; it now stops 27, and the count of games whose board diverged fell from 135 to 117. Any damage card counted against the 135 is an upper bound until it is re-read against the 117 — including the twelve `-damage field 3` cards of `docs/_reports/2026-08-29-empirical-divergence-cards.md`, whose Grass Knot and Heavy Slam rows remain the sharpest open damage question in this document. Nothing in the stage table below was measured on that arm, so no stage verdict here changes.

5.211.0 — THE SPREAD EVERY DAMAGE NUMBER HERE RUNS ON IS ASSIGNED, NOT OBSERVED. A Showdown open team sheet reveals species, item, ability, moves, nature, gender and level and NOT the spread, so `engine/game_differential.js` `spreadFor(index)` assigns one from the body's slot index: 66 points, a 32 cap, a descending Speed ladder by slot, the remainder to the higher attacking stat and then spilling to Sp. Def then Def, and **nothing into HP** — deliberate, because Champions' Showdown line adds the investment plus 75 for HP while medicham2's L50 line has no HP term, so HP points would diverge silently on every body. Both engines are handed the same spread, so a stage disagreement is still a real disagreement; what a clean damage verdict is NOT is a statement about what the ladder actually rolls. `engine/coverage.js` prints this beside the verdict and reads the budget, the cap and the ladder off the driver's source rather than repeating them.

5.210.0 — THE PORY TWO-FEATURE PAIR IS WITHDRAWN: ITS GENERATOR WRITES NO ARTIFACT. `engine/pory_baseline.py` prints a five-arm table and saves nothing, so the material-baseline pair it published on 2026-07-25 never had a source to check it against, and it was scored before that script had a clean-data filter at all. On the clean corpus the comparison is a TIE rather than a loss, measured PAIRED and clustered by game in `data/pory-eval.json`. The withdrawn pair stays in `docs/REVIEW-2026-07-25.md`, the review that measured it. This document does not quote the pair and is unchanged apart from this note.

5.209.0 — STAGE 3 GAINED A MEMBER: FLASH FIRE'S ABSORBED VOLATILE. The blocks below are dated history and are not rewritten. The placement is read off the authority rather than argued: `flashfire.condition.onModifyAtk` / `onModifySpA` return `this.chainModify(1.5)` when the move is Fire and the attacker still has the ability, and `data/mods/champions/abilities.ts` has no `flashfire` key, so mainline's handler is the format's unchanged. It therefore folds into the `_aCh` relay beside Guts, Huge Power, Solar Power, Orichalcum Pulse and Hadron Engine and is spent once — NOT into the final ModifyDamage chain, which is the mistake §2 already measured for Thick Fat and for Water Bubble — the disagreement rates are in that section and are not restated here. The multiplier, the boosted type and the stats are read from `typeImmunity.gain.volatileBoost` in `data/tags.json`; no number is typed.

Version: 5.208.0 — 2026-08-28.

5.208.0 — THIS CHAIN GAINED A MEMBER. STAGE 13 NOW CARRIES THE METRONOME ITEM, AND THE STAGE IS DERIVED FROM THE AUTHORITY RATHER THAN CHOSEN. The two blocks below both begin *"nothing in this chain is touched"*; that is no longer true and the change is recorded here rather than by editing them.

WIRE 158 gave the tag `damageMultOnRepeat` its first consumer. The placement is read off the authority, not argued: `data/items.ts:4022` is `onModifyDamage(damage, source, target, move)` returning `this.chainModify([dmgMod[numConsecutive], 4096])`, and `data/mods/champions/items.ts` carries **no** `metronome` key at all — checked against the mod file, not recalled — so mainline's handler is this format's handler unchanged. `data/mods/champions/scripts.ts:293` spends that event as `runEvent('ModifyDamage', pokemon, target, move, baseDamage)` with `pokemon` the ATTACKER, which is why the member reads the ATTACKER's item and sits beside Life Orb rather than beside the resist berries — those are `onSourceModifyDamage` on the defender. Row 13 of the stage table is updated in place.

THE ORDER INSIDE THE CHAIN CANNOT MATTER FOR THIS MEMBER, AND THAT IS A PROPERTY RATHER THAN A HOPE. The ladder is stored in 4096ths, so every step divided by 4096 is a dyadic rational, `_sdTrunc(m*4096)` returns the step unchanged and `ch4096` introduces no rounding of its own — the same property this file already records for x1.5 and x2. Nothing in the engine types a step: the array and its denominator are read from the tag, an unreadable ladder applies nothing and is counted, and the per-body consecutive-use counter is written at the authority's own `onTryMovePriority: -2` position, below the PP deduction and above the shield gate.

THE DAMAGE DIFFERENTIAL WAS NOT RE-RUN AND THAT IS A GAP, NOT A CLEARANCE. It still reads 0 disagreements of 6000 at each of the sixteen band indices, measured before this member existed. Its declared scope is damage only, and it has never applied a multi-hit move. The evidence for the new member is two dedicated probes instead: a ladder probe pricing rungs 0 to 6, with rung 0 byte-identical to a body holding no item, and a six-turn game probe comparing 394 leaves at each of 7 boundaries. Both were shown red first under `MEDI_NO_METRONOME_LADDER=1`.

5.207.0 — RE-READ AFTER THE CLOSET PASS: NOTHING IN THIS CHAIN IS TOUCHED, AND THAT IS ASSERTED RATHER THAN ASSUMED. The 5.207.0 release closed the last open gate clause by DECLARING one divergence — a Perish Song faint announced above `|upkeep|` instead of below it — through a `kind: 'CLOSETED'` row in `engine/quarantine.js`. **No engine byte moved and no artifact was regenerated.** The damage differential was not re-run because nothing that feeds it changed; it still reads 0 disagreements of 6000 at each of the sixteen band indices, with the same standing scope limit: damage only, and it has never applied a multi-hit move. The stage table below stands unedited.

5.206.0 — RE-READ AGAIN AFTER THE CRLF PASS: NOTHING IN THIS CHAIN IS TOUCHED, AND THAT IS ASSERTED RATHER THAN ASSUMED. The 5.206.0 release restored five withheld gate clauses that had gone blank on a line ending, pinned seventeen frozen sources to LF in `.gitattributes`, and re-ran the roster, the whole-game differential and the staged-mechanics comparison on release `5f3f7141227c`. **No damage stage, no ordering, no rounding rule and no override was changed by any of it.** The damage differential was not re-run because it was never stranded — its gate clause passed throughout — and it still reads 0 disagreements of 6000 at each of the sixteen band indices, with the same standing scope limit: damage only, and it has never applied a multi-hit move. The stage table below stands unedited.

5.205.0 — THE SPRINT IS PAUSED AND THIS FILE IS RE-READ, NOT REWRITTEN: NO STAGE MOVED. The living-docs deferral that ran from 2026-08-10 is over. Across the whole sprint the stage ORDER inside one hit is unchanged, and `tests/test-damage-stages.js` re-reads **1696/1696 exact, 0 at the wrong stage**, across all sixteen rolls and both crit states, with 5 re-derived `CH_EXACT` overrides and 0 wrong. The population moved (this file previously recorded 1728 cells); the verdict did not.

WHAT CHANGED IS OUTSIDE THIS CHAIN, AND BOTH ITEMS BELONG TO THE BATTLE LOOP. The crit is now drawn **once per hit** rather than once per click, which is what the authority does — its loop is the Champions mod's and its die is mainline's, inside the per-hit `getDamage` call, and neither `getSpreadDamage` nor `getDamage` is overridden by the mod. Arrival 2 of a volley no longer inherits arrival 1's crit. Separately, the event die itself gained a finalising mix; before that, consecutive arrivals shared a 16-bucket damage index 89.5% of the time against a correct 6.25%. **Neither touches the stage order this document describes**, and both are why a damage figure measured before 2026-08-27 is void rather than stale.

THE DIFFERENTIAL THAT CHECKS THIS CHAIN HAS NEVER APPLIED A MULTI-HIT MOVE, AND THAT MUST BE SAID WHEREVER ITS FIGURE APPEARS. Read from `data/engine-diff.json`: 6000 compared, 6000 agreed, 0 disagreed, and 0 at each of the sixteen band indices separately. Its own `scope` field limits it to damage only — no items, no abilities, no turn order, no status duration, no switching — and it records `skipped_multihit` 134 and `skipped_ability_multihit` 17, because the harness calls the authority's single-hit entry point rather than the volley loop. The multi-hit defects corrected during this sprint were invisible to it by construction. **The interior of a multi-hit range remains a single draw across the summed endpoints** rather than N independent ones; that is unchanged by this pass and is still the battle loop's question rather than this chain's.

3.98.0 — NO STAGE MOVED AND NOTHING IN THIS FILE CHANGED. ROADMAP #126 wired Quick Guard onto the priority-refusal gate. That gate sits in the TURN LOOP, above the action-kind dispatch, and never reaches `dmgRange`: a refused move deals no damage at all rather than damage at a different stage. `tests/test-damage-stages.js` is unchanged. The version moves because the CHANGELOG top moved, and this line says why the content did not.

3.97.0 — THE DAMAGE IS A LOOP OVER HITS NOW, AND NO STAGE MOVED. `dmgRange` is a wrapper over `dmgRangeOneHit`; the stage ORDER inside one hit is byte-for-byte what this document already describes, and `tests/test-damage-stages.js` re-reads **1728/1728 exact, 0 at the wrong stage**. What changed is how many times that chain is spent: once per HIT for a move whose base power is a function of the hit index (Triple Axel `20 * move.hit`, Beat Up one packet per eligible ally), and once in total — with the old `_hits` scalar — for everything else, including the rest of the multi-hit family. **The pinned endpoints are the authority's**: `min` is every hit at the 85% randomizer and `max` every hit at 100%, which is what a pin produces in Showdown. **The interior of a multi-hit range is still a single draw** across the summed endpoints rather than N independent ones — unchanged by this pass, and it is the battle loop's question rather than this chain's. Fickle Beam's conditional power left this file's arithmetic entirely: it is a DRAW taken in the battle loop, not a ×1.3 on the base power.

3.96.0 — THREE DEAD LINES LEFT THE ATTACK AND DEFENCE CHAINS, AND ONE LIVE ONE JOINED. The hardcoded Choice Band / Choice Specs / Assault Vest multipliers were permanently false — all three are banned in this format — and are replaced by a derived `statMult` consumer whose only live member here is Light Ball. The chains themselves are unchanged in ORDER and `tests/test-damage-stages.js` re-reads **1728/1728 exact, 0 at the wrong stage. The boost-stage difference filed at 3.89.0 is still filed and still not fixed.**

3.95.0 — A DAMAGE ANSWER CHANGED, AND IT IS NOT A STAGE. `dmgRange` now returns **0** against an intact Disguise, because the authority's `onDamage` blocks the move outright and the `maxhp/8` that busts it belongs to the ABILITY. Nothing in the table below moved — no multiplier changed position, and `tests/test-damage-stages.js` re-reads **1728/1728 exact, 0 at the wrong stage** — but this is the first entry here that alters what the function RETURNS rather than where a multiplier applies, so it is recorded as such. **The boost-stage difference filed at 3.89.0 is still filed and still not fixed.**

3.94.0 — NOTHING IN THE TABLE BELOW MOVED. ROADMAP #110's `selfBoost` fix adds the USER's own stat change to two move rows; it is a boost applied after the move, not a multiplier inside it. `tests/test-damage-stages.js` re-reads **1728/1728 exact, 0 at the wrong stage. The boost-stage difference filed at 3.89.0 is still filed and still not fixed.**

3.93.0 — NOTHING IN THE TABLE BELOW MOVED. ROADMAP #110's partial-trap fix is a duration COUNTER, not a multiplier, and the chip fraction it carries (1/8) was correct throughout — what changed is that it is now derived from `onStart`'s `boundDivisor` rather than typed. **The boost-stage difference filed at 3.89.0 is still filed and still not fixed.**

3.92.0 — NOTHING IN THE TABLE BELOW MOVED. `tests/test-damage-stages.js` stopped padding its inert slots with `Tackle`, which is `isNonstandard: 'Past'` and not in this format. Those slots never act, so the reading is unchanged: **1728/1728 exact, 0 at the wrong stage. The boost-stage difference filed at 3.89.0 is still filed and still not fixed.**

3.91.0 — NOTHING IN THE TABLE BELOW MOVED, AND NO STAGE QUESTION WAS RAISED. ROADMAP #116 is a legality guard on the probe harness, not a multiplier. It changes which bodies may be staged, never where a multiplier applies. Worth one line here for a reason that touches this document directly: the stage table is measured by probes, so a probe staging an entity the format does not contain would put a row in it about a mechanic no game can reach. **The boost-stage difference filed at 3.89.0 is still filed and still not fixed.**

3.90.0 — NOTHING IN THE TABLE BELOW MOVED, AND THE ONE STAGE QUESTION THIS PASS RAISED WAS ANSWERED NO. ROADMAP #103 is a hit COUNT, not a stage. The plausible alternative WAS a stage question — whether the authority's per-hit floor (`n` independent `floor`s) differs from this engine's single `floor(v * n)` — and it was ruled out with arithmetic rather than a preference: `roll()` already returns an integer, so for an integer count the two expressions are equal for every value. The multiplication at `dmgRange`'s tail is unchanged. **The boost-stage difference filed at 3.89.0 is still filed and still not fixed.**

3.89.0 — NOTHING IN THE TABLE BELOW MOVED, AND ONE NEW DIFFERENCE IS FILED INTO IT. ROADMAP #101 and #102 are a post-damage REACTION and a HEAL; neither is a stage. But wiring Strength Sap's heal off `getStat('atk', false, true)` surfaced a boost-stage difference that IS a damage question: `dmgRange` applies a stage as `Math.floor(x * boostMul(s))`, where `sim/pokemon.ts` MULTIPLIES on a positive stage and **DIVIDES** on a negative one. The two disagree wherever the float lands just under an integer — at `s = -1, x = 3` they give 1 and 2. The new heal uses a helper (`statWithBoost`) that mirrors the authority exactly; `dmgRange` was deliberately left alone in the same pass, because changing it is a damage change and would have needed its own measurement. **Filed here, not fixed.**

3.88.0 — TWELVE MOVES WERE PRICED OFF GENERIC GEN-9 DATA INSTEAD OF THIS FORMAT'S, AND THE BUILDER THAT FIXED THEM WAS ONE RUN AWAY FROM DELETING TEN SPECIES. Trop Kick read 70 where the format says 85, Mountain Gale 100 against 120 — ours low in all twelve, and MAG's own table had the right numbers the whole time, so the two engines disagreed on every one. Asking what a regeneration WOULD do, before running one, turned up 788 destructive changes waiting in the same builder and a header stamp whose regex had never once matched. `buffsHolderOnHit` also gained its condition by derivation — Anger Point only on a critical hit, Justified only on Dark — but **the engine does not read it yet and nothing behaves differently**, which is said here rather than left to look like a fix.

3.87.0 — THE STAGE WAS RIGHT AND THE INPUT TO IT WAS NOT: TWO READERS OF THE WEATHER. Nothing in the table below moved. The BasePower-chain member at `:1991` (`weatherScaled`) sits at the correct stage and reads the correct sky, because `dmgRange` shadows the field through `effWeatherOf` at its first statement — which applies the PRIVATE weather a Mega Sol body carries. The battle loop's own type authority, `effMoveType`, did not: it read `field.weather` raw. So the same click was a Fire move to this table and a Normal one to the stage-5 immunity gate, and a Meganium-Mega's Weather Ball priced here at 128-151 dealt 0 to a Ghost. `effMoveType` now CALLS `effWeatherOf`. This is §2's lesson in a new place: the stage-by-stage audit can be clean while a DIFFERENT reader of the same fact makes the whole row unreachable.

3.86.0 — EVERY PUBLISHED ARTIFACT HAS A WRITER NOW, INCLUDING THE ARM MILTANK ACTUALLY RUNS. The tool that answers "is this number still true" could only find a writer by an artifact's literal name beside a write call in two directories — so the mechanics census, the game differential, the interaction matrix and the deliberate roster, which are the four clauses of the MEDICHAM gate, had no row at all. Not ok, not unsafe: absent.

The graph goes 115 to 160 artifacts and the unknown set 61 to 16, membership derived through four ranked arms that each record how they matched. UNSAFE rises 13 to 20 because seven artifacts that were always unsafe are now visible; none left the set.

3.85.0 — THE WHOLE SITE WITHHOLDS NOW, AND THE DEPLOYED COPY WAS MISSING THE FILE THAT MAKES IT WORK. Five pages LOADED a quarantined artifact as data instead of quoting its verdict, so the citation checker could not see them at all; seven of the Stadium's fifteen cabinets now go dark, each keeping its seat and its button and answering with the quarantine instead of a number. The file that drives all of it, `quarantine-data.js`, did not exist under `app/` — which is the copy a visitor loads — so every guard there took the healthy path. That is the same failure as the day before, one directory over.

3.83.0 — THE PINCH FAMILY FIRES, AND THE REFUSAL THAT HID IT WAS CORRECT THE WHOLE TIME. 3.80.0 (below) found that Blaze, Torrent, Overgrow and Swarm carry their below-1/3-HP condition as PROSE and that the `damageBoost` consumer refuses any condition it cannot evaluate. That refusal is #92's own rule and it is not removed. What changed is upstream: `engine/tag_dex.js` now derives the gate by SHAPE out of Showdown's `attacker.hp <= attacker.maxhp / 3` into `{cond:'hpFraction', of:'self', cmp:'<=', num:1, den:3}`, and `condHolds` evaluates it in INTEGER arithmetic — `maxhp * (1/3)` is not `maxhp / 3`, and a body at exactly one third would be refused a boost it is owed. An `onlyWhen` the engine still cannot read returns null, still refuses, and is now counted. The narrowed shape's membership went 5 → 9 and `tests/test-damage-stages.js` still reads 1728/1728 exact with 0 at the wrong stage. 9,141 uses, on today's corpus.

ROADMAP #88 AND #91 — ONE PIN WAS ONE CORNER, AND A CLICK WAS COUNTED AS A TEST (3.73.0). Every die in the differential was pinned a single way, which bought determinism — any difference is a bug, no statistics — and paid for it in coverage nobody had priced. The speed tie always resolved the same direction, every move below 100 accuracy MISSED ON BOTH SIDES, and damage was always the maximum roll, which is the one roll where the crit's wrong position happened to come out right. Rock Slide had never connected in this instrument; under the new arms it misses in one and hits in another, and a crit lands in the bottom arm and not the top. The pin set is now a declared run parameter, digested into `mode`, and a before/after pair whose pins differ is REFUSED rather than reported. Separately, coverage credit moved from the CLICK to the OBSERVED EFFECT: the old rule incremented when an entity was clicked and never asked whether the move did anything, so Haze clicked into a board with no boosts on it — a no-op — marked Haze exercised and stopped the steering selecting it. Five rows were clicked-or-present and did nothing at all: `critDamageUp`, `preventsSwitch`, `privateWeather`, `clearsScreens` and `preTurnShield`. The old rule called all five covered. **THE BASELINE IS RESET: both changes alter which games get played, so no run after this is comparable with the turn-1 figure published at 3.71.0 or with `data/state-ladder.json`.** And an ENGINE defect fell out of the tie work, filed rather than fixed here: the two engines have disagreed about EVERY speed tie for the life of this instrument — the authority resolves a tie to the LATER body in input order, `sortTurnOrder` draws one tie value per action from a constant scalar so the sort is stable and takes the EARLIER one. The instrument's own header claimed the pin made them agree by construction; that claim was false and was repeated as fact before it was checked. `sortTurnOrder` is the live engine, not instrument code.

3.82.0 — THE FIRST ENGINE FIX OF THE QUEUE LANDS: THE VOLATILE DURATION FAMILY, 9,092 USES, AND IT WAS THE PERISH SONG BUG A SECOND TIME. Showdown decrements a volatile's duration inside the Residual event, so one applied on turn N has already spent a turn by the end of it. That defect was documented in this engine for Perish Song, fixed for Perish Song, and left standing for every other duration-bearing volatile. Taunt and Disable now match the official engine; Encore's counter row is gone and only a separate HP row remains. Whole-game board agreement rose 76.9% to 78.9% on a paired differential, the roster's moves queue fell from 52 to 50, and the census did not move. The previously published baseline could not be reproduced because the census digest and the team store had both shifted underneath it — so the run was discarded and re-taken paired. The delta is the measurement.

3.81.0 — THE QUARANTINE REACHED THE BOARD, AND THE CHECKER THAT POLICES IT WAS BLIND TO THE PUBLISHED COPY. Thirteen slots on ABRA WORLD's status board now render as a redaction bar rather than a number, each carrying the artifact, the reason and the command that re-runs it. Two defects in the guard itself were found doing it: its own selftest had gone red — an `all`-stage artifact was matching ANY requested stage name, so "a missing stage must FAIL" stopped being enforced — and its citation walker looked at docs/ and web/ but never at app/, which is the copy a visitor actually loads. Five withheld verdicts were being published from app/ the whole time the check read green.

3.80.0 — THE DELIBERATE ROSTER'S INERT BUCKET COLLAPSED BY 94.1% OF ITS USAGE, AND A FAMILY OF ABILITIES WORTH 8,524 USES TURNS OUT NEVER TO HAVE FIRED. 124 abilities were falling through a catch-all that stages a plain attack, so the condition each one needs was never created and the roster honestly reported INERT — which reads as "nothing to test" when the truth is "never tested". Fifteen new shape rules take that bucket to 59 abilities / 4,261 uses, all 22 ability rules caught their own break, and nothing left in the bucket is above 500 uses. The first real defect it found: Blaze, Torrent, Overgrow and Swarm carry their below-1/3-HP condition as PROSE, and the consumer refuses any condition it cannot evaluate — so the pinch family has never once fired. The roster's artifacts are now written per stage, so the quarantine gate reads measurement rather than absence.

3.79.0 — EVERY FIGURE DOWNSTREAM OF MEDICHAM IS NOW WITHHELD RATHER THAN CAPTIONED, AND THE DELIBERATE ROSTER'S MOVES STAGE RAN FOR THE FIRST TIME. Will's standing call: *"all engines that take medicham's output should be regarded as out of date and we should stop referencing them until medicham is up to date and we can rerun them."* 34 of 114 artifacts are downstream and are no longer printed at all — R1, R2, R3, R4, leaf calibration and the weights among them — because a caption is not a quarantine: `PRE-CHANGE` had been printed beside those numbers for days and they went on being quoted anyway. Membership is derived from the dependency graph, not typed, and the gate that lifts it is computed from the differential and the roster, where a MISSING stage counts as failing. The roster gained 26 move shape rules and staged all 500 legal moves for the first time, returning a 79-row queue over ~15,000 uses. Its control arm was found to be measuring the CONTROL rather than the subject, which made six ability findings false; **Weather Ball, Sand Rush and Damp are retracted as defects and are correct.**

3.78.0 — THE SHEET'S REAL NATURE NOW REACHES BOTH ENGINES, AND THE TURN-1 NUMBER FELL. THAT IS THE INSTRUMENT GETTING HONEST. The whole-game differential built every body `Serious` while the stored sheet beside it said `Modest`, and with every body flat AND Serious, 326 of 357 species in the format share a Speed with some other species — so the rig MANUFACTURED speed ties and almost never tested a real speed differential. Carrying the declared nature cut the tied groups the resolver has to break from 348,595 to 243,467 over the same 1,998 games, a 30.2% fall. The instrument's own numbers went DOWN, as predicted before the run: the board at the end of turn 1 is identical in 97.4% of games flat against 97.3% natured, games whose board never parted 80.8% against 78.8%, and the median turn of first board divergence one turn earlier at 7. Encore divergences nearly doubled (10 to 19 games), which is what a duration volatile that only bites when turn order does looks like when turn order starts being tested. THE SPREADS REMAIN ABSENT AND ALWAYS WILL BE — a Showdown open team sheet does not reveal them (`"evs": null` on 173,784 of 173,784 stored bodies), so this narrows the declared gap and does not close it. Neither engine is told the other's answer: both are told the nature and each computes, and the alignment assertion still reads 0. It read 21 on the first run and all 21 were Ditto — entry-time Imposter had already transformed the medicham body, and the harness was writing the copied stat line onto Showdown's Ditto before the game began. Census 324 live, 0 missing, unchanged.

3.77.0 — CONFUSION DID NOT EXIST, AND BURN HAD NEVER BEEN ON A BOARD. The confusion volatile was written and never read or ticked, so Hurricane's secondary — 3,779 uses — fell through every branch, and the two berries that clear confusion looked dead because there was nothing to clear. The sleep counter was an ordering bug: the authority runs sleep before flinch, ours ran flinch first, so a body that was asleep AND flinched never ticked and woke a full turn late. Burn, by contrast, is CORRECT and was confirmed rather than changed — but it had never once been staged, because Will-O-Wisp is 85-accurate and the harness pin makes every sub-100 move miss. The freeze timer is correct too; what was missing was the instrument, which carried no freeze counter at all, so the engine's value could drift and no measurement would see it. Census 324 live, 0 missing.

3.77.0 — THE ACROSS-A-SWITCH ARM FOUND A DEFECT ONE DAY OLD THAT FIXING SOMETHING ELSE CREATED. A transform never reverts when the body leaves the field: the authority clears it in `clearVolatile`, and this engine sets the flag and never unsets it. Since the transform also overwrites the body's name, stats, types, moves, boosts and ability, a benched Ditto is PERMANENTLY the thing it copied — so the two engines then choose replacements from benches that no longer describe the same Pokemon, and worse, a Ditto can only ever transform ONCE PER BATTLE, because the guard refuses a second. Re-copying is the entire function of the Pokemon. Imposter first fired the day before, and the out-and-back scenario that exposes this only became expressible hours earlier. The roster's two owed arms — across a switch, and at the exact HP line — are both built, both red-demonstrated, and Speed Boost and Focus Sash both match.

3.77.0 — A STAGED SCENARIO CAN NOW SWITCH, SO A MID-TURN ENTRANT IS EXPRESSIBLE FOR THE FIRST TIME. The scenario driver understood only a move; every other step became a pass, so no staged test could put a body on the field part-way through a turn. That single gap blocked three things at once: Speed Boost's entry gate, which exists only for a body that just switched in; Hunger Switch's flip and Zero to Hero's switch-out transform; and the whole across-a-switch arm of the roster. Four of the six engine defects found the day before were about a MOMENT rather than an effect, and no scenario without an entrant can express one. Verified end to end: Espathra switches in and reads +0 Speed in both engines on the turn it arrives, then +1 at the end of the next, with all 131 fields identical on both boundaries.

3.77.0 — ALL 316 ABILITIES STAGED DELIBERATELY, AND A FREE +6 ATTACK FELL OUT. Anger Point and Justified are one defect twice: a conditional boost-on-being-hit whose condition is never checked, so Anger Point grants +6 Attack off an ordinary hit where it requires a crit, and Justified grants +1 off a Poison move where it requires Dark. Hustle applies no 1.5x Attack at all. Two facts about the instrument matter as much: Gastro Acid does not suppress an ability here, and since suppression is the ONLY control available to 23 abilities, checking that control against a known-live fixture is what stopped five more from being published as dead for the control's failure rather than their own. And a fact about the regulation rather than the simulator: 113 of 316 legal abilities have NO legal carrier, so the effective roster of this format is about 203.

3.77.0 — FIVE MECHANICS THAT DID NOTHING, AND ONE THAT WAS ALREADY RIGHT. Imposter never transformed Ditto; Hunger Switch never flipped Morpeko; Knock Off took its 1.5x against an item it cannot remove; Fling never became an attack at all, because a base power of 0 made the click fail a `hasPower()` gate; and Roar's phaze branch held a Pokemon-first target, so a phaze after a pivot dragged nobody — the SIXTH site missed by the slot-first sweep, and at priority -6 the worst possible place to hold a body rather than a slot. Mawile's mega ability swap, which had been blamed for a whole family of Attack-stage divergences, WAS ALREADY CORRECT: the scenario was board-identical on its first run, and deleting the swap deliberately parts two fields at once, so the symptoms were real symptoms of a bug this engine does not have. Census 319/319 live, 0 missing; the staged harness now carries 24 scenarios, all clean and all breakable.

3.77.0 — THE INSTRUMENT RESOLVED A SWITCH BY TWO DIFFERENT KEYS AND FAILED SILENTLY BOTH WAYS. The driver names a bench member by Showdown's species id; the Showdown side looked it up by species id and the medicham side by the body's DISPLAY NAME. Those agree until a body is renamed — which this engine began doing the day before, when Disguise started renaming a busted Mimikyu, Zero to Hero started renaming Palafin, and Hunger Switch was queued to flip Morpeko every turn. After a rename the two keys part and that body can never be switched to again. Neither side raised anything: an unresolved lookup answered `pass` on both, so one engine could switch while the other stood still, producing a different board with no evidence attached. The key is now stamped at build time from the same expression the driver uses, and a miss is counted and printed beside the other declared gaps (0/0 over 120 games). This is an INSTRUMENT change rather than an engine one, so it alters what a measurement sees; it was also LATENT UNTIL THE FORME FIXES LANDED, and the deliberate-roster build would have walked into it.

> ***NO STAGE MOVED IN 3.77.0, AND THE VERSION MOVED ANYWAY — the reason is worth stating rather than pinning.*** *WIRE 139 changed WHICH BODY a move resolves against (the slot, not the Pokemon, which is what `Battle#getTarget` does), and WIRE 140 added Ally Switch, which moves two bodies between slots mid-turn. Neither touches a multiplier or its stage, so every row in the table below still holds exactly as measured — but both sit UPSTREAM of the whole table: a multiplier applied at the right stage to the wrong defender is wrong for a reason this document cannot see. Said here so a later reader does not conclude the audit was re-run.*

> ***WIRE 133–138 ADDED ONE MULTIPLIER TO THIS AUDIT AND SETTLED THE SPEED-TIE PARAGRAPH ABOVE (3.74.0).*** *The paragraph at the head of this file was RIGHT that the two engines disagreed about every speed tie and RIGHT that `sortTurnOrder` is the live engine, and its DIAGNOSIS was incomplete: "the authority resolves a tie to the LATER body in input order" is what the authority PRODUCES UNDER THIS HARNESS'S PIN, which replaces `PRNG.shuffle` with a no-op — it is not a rule. The rule is `Battle#speedSort`, a SELECTION SORT whose swaps move UNTIED elements around, ending in a Fisher-Yates over the tied group: a speed tie is a COIN FLIP. The engine now performs the same selection sort and resolves the residual group with the per-action uniform key it already drew, so both engines land on the same body under identical pinned dice and both are a fair coin under real ones. `tests/test-speed-tie.js` proves it in both team orientations, with the tied pair on one side, on a three-way tie, and against a no-tie control.* ***The one change to the DAMAGE formula itself is WONDER ROOM*** *(`swapsDefences`, 11 uses), which had no consumer at all. It swaps the STORED DEFENSIVE STAT and NOT the boost stage — `Pokemon#getStat` swaps `storedStats` at the top of the function and then applies `boosts[statName]`, the ORIGINAL stat's stage — so the swap is applied to `D` before the stage multiplier and `_dKey` is deliberately left unrewritten. It rides on whichever defence the move attacks into, so Psyshock and Body Press inherit it through `statSwap` rather than through a second rule. Nothing else in §2a or §3 moved.*

> ***THIS AUDIT HAS BEEN LANDED. ROADMAP #92, 3.73.0.*** *Everything §6 lists as open work is fixed, and the numbers below are now HISTORY — they describe the engine at release `dc3c43336539`, not the engine in the tree.* ***Read §2a and §3 as the record of what was wrong, not as a description of what is wrong.*** *What replaced them: - every `onBasePower` member is folded into ONE relay spent once, and every `onModifyAtk` / `onModifySpA` / `onModifyDef` / `onModifySpD` member into two more — the STAGE and the CHAIN halves of §2's finding, which had to move together; - Friend Guard is inside the ModifyDamage chain rather than beside it; Helping Hand and the ally multiplier reach `dmgRange` on a seventh argument, because what it cannot DERIVE it can be TOLD; - the rolled crit's x1.5 is applied inside `dmgRange` before the randomizer, where the authority applies it, and Sniper has left that multiply for the final chain; - the four field terrains exist for the first time, with the authority's own grounded subject.* ***The claim is checked rather than asserted:*** `tests/test-damage-stages.js` *runs 54 scenarios × 16 damage rolls × 2 crit states against Showdown's own `moveHit` and demands EXACT equality —* ***1,728/1,728*** *— and it was shown RED on two deliberate reversions before being trusted. The census carries five of these*

> *as probes (* `move|setsTerrain` *×4,* `ability|damageBoost` *), which is what the last paragraph of this header said it did not yet do.* ***§2c is the part that keeps its original force****: it is the list of things that were checked and found CORRECT, and the gate now re-checks every one of them on every run. Four things are still not fixed and each is named with its reason in* `docs/ENGINE.md` *— Charge (no volatile exists to read),* `terrainScaled` *'s grounded SUBJECT (the tag carries none), Rivalry (no gender in* `MC.mons` *), and the artifact storing 1.3 as a float where the authority spells* `[5325,4096]` *(the engine carries a four-entry override that the gate re-derives from the live dex).*

Audited against engine release `dc3c43336539` and the Showdown checkout at `SHOWDOWN_PATH` . Every rate in this document was measured in the session that wrote it.

Will, 2026-08-07: *"LETS CHECK THE DAMAGE FORMULA FOR ALL ITS COMPONENTS AND COMPARE OURS AGAINST SHOWDOWN."*

Read against the frozen release `dc3c43336539` , not the live tree — `engine/medicham2-browser.js` was being edited by another agent while this was written. Every line number in the "ours" column is a line in `data/releases/dc3c43336539/engine/medicham2-browser.js` . Every line number in the authority column is `pokemon-showdown/sim/battle-actions.ts` or `sim/battle.ts` .

This document is an AUDIT. It changes no engine file and lands no mechanic. Nothing here is a probe in `tests/test-mechanics.js` yet, so nothing here is carried by the census — that is the next pass's job.

---

0. THE ANSWER THAT WAS ASKED FOR FIRST — FAIRY AURA IS A BASE POWER MULTIPLIER

`pokemon-showdown/data/abilities.ts` — `fairyaura.onAnyBasePower` , priority **20**:

```
if (target === source || move.category === "Status" || move.type !== "Fairy") return;
if (!move.auraBooster?.hasAbility("Fairy Aura")) move.auraBooster = this.effectState.target;
if (move.auraBooster !== this.effectState.target) return;
return this.chainModify([move.hasAuraBreak ? 3072 : 5448, 4096]);
```

Three facts, all read out of the handler rather than remembered:

1. **The stage is** `BasePower` **, not** `ModifyDamage` **.** Dark Aura is the same handler with `"Dark"` .
1. **The multiplier is** `[5448, 4096]` **, not** `1.33` **.** Our tag artifact carries `auraBoost.mult = 1.33` , and `trunc(1.33 * 4096) = 5447` — one 4096th low. Pass the pair to `md4096` / `ch4096` , which already accepts `[num, den]` for exactly this reason (its own header records the same trap for Tough Claws, `1.3 -> 5324` against the authority's `[5325, 4096]`).
1. **Aura Break does not suppress the aura — it INVERTS it** to `[3072, 4096]` (x0.75) at the same stage. Measured: Sylveon Moonblast into Goodra, no aura 98, Fairy Aura 132, Fairy Aura + defender Aura Break **74**. `aurabreak` has no entry in the tag artifact at all.

The cost of landing it at the wrong stage, measured

Eight Fairy-move rows, bodies flat L50/0EV/31IV/Serious in both engines, top roll. Two candidate fixes scored against Showdown: the multiplier applied to BASE POWER, and the same multiplier spent on the FINAL damage.

row	Showdown	ours today (no aura)	fix at BasePower	fix at ModifyDamage
Sylveon Moonblast -> Goodra	132	98	**132**	130
Sylveon Dazzling Gleam -> Snorlax	72	55	**72**	73
Gardevoir Moonblast -> Snorlax	94	72	**94**	96
Clefable Play Rough -> Goodra	162	122	**162**	162
Azumarill Play Rough -> Snorlax	67	51	**67**	68
Sylveon Draining Kiss -> Goodra	72	54	**72**	72
Gardevoir Dazzling Gleam -> Goodra	122	96	**122**	128
Clefable Moonblast -> Heliolisk	85	66	**85**	88

BasePower-stage fix: 8/8 correct. ModifyDamage-stage fix: 2/8. The two candidate fixes agree with each other on 2/8, so a wrong-stage aura is not "close enough" — it is wrong three quarters of the time, by up to 6 points on a 122-point hit.

The usage argument, and `uses` **is a sheet count so it is read carefully.** The tag artifact says `fairyaura: uses 0` . That figure is worthless here for the reason `docs/LESSONS.md` 3 gives: the ability is Gardevoir-Mega's, so it never appears in a sheet's ability slot. The real exposure is the STONE, times **every Fairy move either side clicks while it is on the field** — `appliesToEveryone: true` in our own artifact.

Exposure is read live from `data/tags.json` and is deliberately NOT restated here. The counts that sat on this line were true when it was written and were stale within hours — `tags.json` is regenerated whenever the tagger runs, and every one of them had moved by the next gate run. What matters and does not drift: the stone is on the order of hundreds of sheets, Moonblast is the most-clicked Fairy move in the corpus by a wide margin with Dazzling Gleam second, and the aura applies to EVERY Fairy move on the field rather than only the holder's — so the exposure is the stone's sheets multiplied by every Fairy click by anyone, which is why `fairyaura: uses 0` was worthless evidence. Read the current numbers out of the artifact.

---

1. THE AUTHORITY'S PIPELINE, ONE ROW PER STAGE

`getDamage` (`sim/battle-actions.ts:1585`) then `modifyDamage` (`:1724`). Three ways the authority applies a multiplier, and the difference between them is where every finding in this document lives:

how	what it is
`modify(v, m)`	`sim/battle.ts:2329` — `tr((tr(v * tr(m*4096)) + 2047) / 4096)` . Fixed point, round-half-up on 4096ths.
`chainModify` / `runEvent`	`battle.ts:2318` / `2302` — each handler folds into `event.modifier` ; the chain is spent ONCE by `finalModify` -> `modify` . **Two handlers in one chain truncate once, not twice.**
plain `tr()`	a bare truncated multiply. The source labels two of these "not a modifier" **on purpose**.

#	stage	authority (line)	how	ours (frozen line)	verdict
1	**BasePower chain** — items, abilities, terrain, Helping Hand, Charge, auras, the move's own `onBasePower`	`battle-actions.ts:1650` `runEvent('BasePower')`, then `clampIntRange(bp,1)` at `:1653`	chainModify, spent once	scattered: `:1985` (-ate), `:1991` (weatherScaled), `:1997`, `:2008-2112` (variablePower), `:2121-2137` (conditionalPower)	**PARTLY SAME STAGE** — the move-side members are here and correct; every ITEM and ABILITY member is not (see §2)
2	**Tera 60-BP floor**	`:1660-1667`	assignment	absent	ABSENT — no Tera in this engine at all; out of scope, stated
3	**stat modifiers** `ModifyAtk/SpA/Def/SpD`	`:1708-1709`	chainModify, spent once	`:2192-2245` through `md4096`	**SAME STAGE** for Choice items, Guts, Solar Power, Orichalcum, Hadron, the four Ruin abilities, sand/snow defence, **Flash Fire's absorbed volatile** (added at 5.209.0). **WRONG STAGE** for Thick Fat / Heatproof / Purifying Salt / Water Bubble (see §2)
4	**base damage** `tr(tr(tr(tr(2L/5+2)*bp*A)/D)/50)`	`:1718`	integer	`:2246` `Math.floor(Math.floor(22*mvBP*A/D)/50)+2`	**SAME** — at L50 `2L/5+2 = 22`, and `22*bp*A` is already an integer so the extra `tr` is a no-op
5	**+2**	`:1731`	addition	folded into `:2246`	SAME
6	**spread x0.75**	`:1737`	`modify`	`:2247` `md4096(base, 0.75)`	**SAME STAGE, SAME ARITHMETIC.** Measured: spread arm 13/282 disagree, control 13/282 — the same rows. Spread adds zero error
6b	**Parental Bond** (2nd hit x0.25)	`:1742`	`modify`, per hit	`:2337-2341` one hit x1.25 at the base stage	STRUCTURALLY DIFFERENT — declared in the engine's own comment; a two-hit move rolled once is not this stage's problem
7	**weather** `WeatherModifyDamage`	`:1746`	priorityEvent -> chainModify	`:2250-2251` `md4096`	**SAME STAGE.** Charizard Flamethrower -> Snorlax: 61 clear, 91 sun, 30 rain, both engines
8	**CRIT — a plain** `tr(x * 1.5)` **, NOT a modifier**	`:1748-1752`	`tr()`	`:2276-2281` (certain crits) / `:5912-5918` **(rolled crits, at the hit site)**	**SPLIT — see §3.** dmgRange's certain crit is at the right stage and passes. The battle loop's rolled crit is applied AFTER everything
9	**the randomizer — also NOT a modifier**	`:1755`, `battle.ts:2388` `tr(tr(d*(100-random(16)))/100)`	`tr()`	`:2528` `Math.floor(base*r/100)` inside `roll()`	**SAME POSITION, SAME ARITHMETIC.** Different SHAPE (11 uniform integers vs 16 inverted indices) — already documented in `engine/game_differential.js`; the POSITION is not also different. Both endpoints match exactly
10	**STAB** (+ `ModifySTAB` for Adaptability)	`:1789-1792`	`modify`	`:2391-2392`, applied at `:2529` `md4096(d, stab)`	**SAME STAGE.** Adaptability x2 via `stabBoost` agrees (132 vs 132)
11	**type effectiveness**, clamped -6..6, `x2` per step up / `tr(/2)` per step down	`:1796-1812`	literal	`:2530` `Math.floor(d*eff)`	**SAME.** `floor(d/4) === floor(floor(d/2)/2)` for every integer, so the single floor is the reference. No clamp in ours, but no move reaches +-7 steps
12	**burn x0.5**, physical, not Guts, not Facade	`:1816-1820`	`modify`	`:2405-2406`, applied at `:2531` `md4096(d, burn)`	**SAME STAGE.** Measured burn arm 8/160 (5.0%) against a control of 4.6% — inside the control's own residual
13	**ModifyDamage chain** — the final item/ability chain	`:1826`	chainModify, spent once	`:2418-2419` `mod` / `MODMUL`, spent at `:2533` `mdChain`	**SAME STAGE AND GENUINELY A CHAIN.** Life Orb, **Metronome** (added at 5.208.0, WIRE 158 — attacker-side, `onModifyDamage`), Expert Belt, resist berries, Multiscale/Filter/Solid Rock/Prism Armor/Ice Scales/Punk Rock-defensive, Tinted Lens, Neuroforce, screens. Measured at the control's residual — see §4
13b	**Friend Guard** (`onAnyModifyDamage`)	`:1826`, same chain	chainModify, **in the same chain**	`:5892-5898`, `md4096` on the already-spent number	**RIGHT STAGE, WRONG CHAIN** — see §3
14	**bypassProtect x0.25**	`:1830`	`modify`, after the chain is spent	`:5931` `md4096(dmg, 0.25)`	SAME — a separate spend is correct here
15	**minimum 1**	`:1838`	`return 1`	absent	ABSENT, **and never observed**: 600 random (attacker, move, defender) draws produced 0 rows where Showdown floored to 1 and we returned 0. Recorded, not ranked
16	**16-bit truncation**	`:1841` `tr(baseDamage, 16)`	`tr()`	absent	ABSENT and unreachable — needs a damage above 2^16

2. EVERY MULTIPLIER WE APPLY, CLASSIFIED BY THE AUTHORITY'S STAGE

This is the table that outlives the audit. **Stage read from the handler's own event name via `Dex.forFormat('gen9championsvgc2026regmb')`, never from memory.**

The `uses` column is the tag artifact's sheet or click count. It is a **prior, not truth**, and it is read from the LIVE artifact while the engine bytes are read from the frozen release — deliberately, because usage is a fact about the corpus and not about the engine, and the corpus has grown since the release was cut. (The frozen copy's figures are lower across the board; nothing in the ranking moves.)

2a. WRONG STAGE — we apply it later than the authority does

Ordered by exposure — class size times usage, not usage alone, which is why eighteen items at a few thousand sheets each outrank one ability at 678. The measured column is the disagreement rate against Showdown over random (attacker, move, defender) triples with flat bodies, top roll, rows dropped when the reference KO'd (clamped) or dealt 0. **The control — the same rows with nothing switched on — disagrees on 4.1% (12/294).** That is the floor: anything at 4% is adding nothing, anything at 35% is the stage.

PER-ITEM USAGE FOR THE TYPE-ITEM ROW IS READ LIVE FROM `data/tags.json` AND IS DELIBERATELY NOT RESTATED IN THE ROW. The counts that used to stand there were correct when written and had all moved by the next gate run, because the tagger regenerates that artifact against a corpus that grows hourly — roughly a quarter of all tagged entities' `uses` moved in the single regeneration of 2026-08-10 (the exact tally is in `docs/MEDICHAM-SPRINT-NOTES.md`, where it is stated once). Membership is what matters and it is derived, not typed.

(The citation moved out of the table row on 2026-08-10, and the reason is worth one sentence. A table row is its own paragraph to `tests/test-docs-current.js`, so naming `data/tags.json` inside the row was a promise that every figure in that row came from it — and the row's other figures are DISAGREEMENT RATES from the run described above, which no artifact holds. `65.0%` passed the citation check for months purely because some unrelated `0.65` sat in `tags.json`, and it stopped passing the moment that number moved. The measurement is unchanged; only the false citation is gone.)

multiplier	authority event	ours (frozen line)	uses	measured disagreement
the 18 type items — every member of `damageMultType`, headed by Fairy Feather, Black Glasses, Mystic Water and Charcoal, with a long tail down to Silver Powder	`onBasePower` x1.2	`:2499-2500` `damageMultType` in the ModifyDamage chain	**the biggest class here**	Black Glasses **65.0% (13/20)**, Charcoal **40.0% (10/25)**
Tough Claws	`onBasePower` `[5325,4096]`	`:2313-2320` `boostsMoveClass`, at the base stage	627	**34.0% (54/159)**
Technician	`onBasePower` x1.5, priority 30	`:2308`, at the base stage	678	**40.3% (31/77)**
Sharpness	`onBasePower` x1.5	`:2313-2320`	314	**48.0% (12/25)**
Sheer Force (the x1.3 half)	`onBasePower` `[5325,4096]`	`:2326-2332` `removesOwnSecondaries.powerMult`	176	staged rows agree; same class, same fix
Thick Fat / Heatproof / Purifying Salt	`onSourceModifyAtk` / `onSourceModifySpA` x0.5 — the **STAT** stage	`:2487-2493` `halvesTypeDamage.attackerStatMult` in the ModifyDamage chain	136 / 17 / 60	**73.1% (19/26)**
Water Bubble (attacking x2 on Water)	`onModifyAtk` / `onModifySpA` — the **STAT** stage	`:2494` in the ModifyDamage chain	131	**77.3% (17/22)**
Iron Fist	`onBasePower` `[4915,4096]`	`:2313-2320`	114	9.5% (2/21)
Dry Skin (x1.25 taken from Fire)	`onSourceBasePower` x1.25	`:2487-2493` `halvesTypeDamage.basePowerMult`	133	**40.0% (10/25)**
Supreme Overlord	`onBasePower`, table `[4096,4506,4915,5325,5734,6144]`	`:2249` `boostsFromFallen`, `md4096(base, 1+0.1n)`	84	staged rows agree at n=1,3; the table is exact 4096ths and `1+0.1n` is not
Helping Hand	`onBasePower` `chainModify(1.5)`	`:5902` `Math.floor(d * 1.5)` on the rolled range, at the hit site	**4306**	**5/5 rows wrong** — Alakazam Psychic -> Snorlax: Showdown 108, ours 109; Kingambit Kowtow -> Snorlax 154 vs 157; Pikachu Thunderbolt -> Snorlax 49 vs 51
Expanding Force / Rising Voltage	`onBasePower` `[5325,4096]`	`:2303-2307` `terrainScaled`, `Math.floor(base*mult)` at the base stage	204 / 123	same class; also a plain float multiply rather than 4096ths
Muscle Band / Wise Glasses	`onBasePower` `[4505,4096]`	`:2516-2517`, hardcoded names in the ModifyDamage chain	112 / 33	Muscle Band **39.6% (74/187)**, Wise Glasses **42.2% (43/102)**
Mega Launcher / Strong Jaw / Punk Rock-offensive	`onBasePower`	`:2313-2320`	29 / 6 / 0	Strong Jaw 25.0% (2/8), Punk Rock 20.0% (1/5)
the -ate abilities' x1.2 (Pixilate 2875, Refrigerate 9, Aerilate/Galvanize/Dragonize/Normalize 0)	`onBasePower` `[4915,4096]`, priority 23	`:1985` `Math.floor(mvBP * damageMult)` — **right stage, wrong rounding** (floor vs round-half-up on 4096ths)	2875	not measurable through `moveHit` (see §5); the retype half is at the right stage and works
Sniper	`onModifyDamage` x1.5	`:2279-2280` and `:5916-5917`, folded into the crit's plain multiply	50	**34.8% top roll, 54.1% bottom roll**

The arithmetic, in full, for the row that started this

Kingambit Kowtow Cleave (Dark, physical, 85 BP) into Charizard. Bodies flat in both engines: atk 155, def 98, Charizard maxhp 153. Top roll, no crit, no burn, nothing else on the field.

```
SHOWDOWN   bp 85
           Black Glasses  modify(85, x1.2)   -- the BasePower chain           = 102
           base           tr(tr(tr(22 * 102 * 155)/98)/50) + 2                = 72
           randomizer     tr(tr(72*100)/100)                                  = 72
           STAB           modify(72, x1.5)                                    = 108
           type Dark vs Fire/Flying = 1x, ModifyDamage chain empty            -> 108

OURS       bp 85          (Black Glasses is not read here at all)
           base           floor(floor(22 * 85 * 155 / 98) / 50) + 2            = 61
           roll           floor(61 * 100 / 100)                               = 61
           STAB           md4096(61, 1.5)                                     = 91
           type 1x, burn 1x
           ModifyDamage   the x1.2 lands HERE: mdChain(91, ch4096(4096, 1.2))   -> 109
```

Same multiplier, same fixed-point helpers, **one stage apart, and a point of damage out**. The x1.2 applied to 85 gives 102, which is a base power; applied to 91 it gives 109, which is a damage. In between sit `tr(.../98)` and `tr(.../50)`, and neither commutes with a multiply.

Two failure modes hide inside "wrong stage", and both need fixing together.

1. **The stage itself.** A base power passes through `tr(.../D)` and `tr(.../50)` before it becomes damage; a final multiplier does not. The truncations do not commute.
2. **The chain.** Showdown folds every `onBasePower` handler into ONE `event.modifier` and spends it once. Ours applies each as its own `Math.floor`. Measured directly: Gallade Drain Punch -> Snorlax with **Iron Fist + Muscle Band** — Showdown 228, ours 227, while each alone agrees. A fix that moves these to the base-power stage but keeps one floor per member will still be wrong when two co-occur.

2b. ABSENT — we apply nothing at all

multiplier	authority event	evidence	usage
field terrain damage — Electric Terrain x1.3 on Electric, Psychic Terrain x1.3 on Psychic, Grassy Terrain x0.5 on Earthquake/Bulldoze/Magnitude, Misty Terrain x0.5 on Dragon	`onBasePower` `[5325,4096]` / `chainModify(0.5)` on the terrain CONDITION	grep of the frozen file: the only terrain reads in `dmgRange` are Hadron Engine (`:2230`) and the per-move `terrainScaled` tag (`:2305`). Measured: Pikachu Thunderbolt -> Snorlax **43 vs 34**; Hatterene Psychic -> Snorlax **94 vs 73**; Garchomp Earthquake in Grassy **60 vs 118**; Dragon Claw in Misty **92 vs 165**	Psychic Terrain 128 clicks + Psychic Surge 2; Electric 11; Grassy 11; Misty 9. Small today, and it is the whole of what a terrain team does
Fairy Aura / Dark Aura / Aura Break	`onAnyBasePower` `[5448,4096]` / `[3072,4096]`	`grep auraBoost` = 0 hits in the frozen engine, and no `aurabreak` entry in the tag artifact	Gardevoirite 412 sheets x every Fairy click on the field. **§0**
Charge (x2 on the user's next Electric move)	`onBasePower` `chainModify(2)` on the volatile	Pikachu Thunderbolt -> Snorlax with Charge: **66 vs 34**	move 1 click, but Electromorphosis applies it too
the whole `damageBoost` tag — Steelworker, Transistor, Dragon's Maw, Rocky Payload, Stakeout, Analytic, Reckless, Rivalry, Flare Boost, Toxic Boost, Sand Force, Gorilla Tactics, Hustle	`onBasePower` (Analytic, Reckless, Rivalry, Sand Force) or `onModifyAtk/SpA` (Stakeout, Steelworker, Transistor, Dragon's Maw, Rocky Payload, Hustle, Gorilla Tactics)	`grep damageBoost` in the frozen engine returns **one hit, and it is inside a comment**. 44 abilities carry the tag; nothing reads it	Reckless 77, Rivalry 39, Analytic 14, rest 0 on this corpus
Battery / Power Spot / Steely Spirit (ally base-power boosts)	`onAllyBasePower`	absent from the engine; Battery and Power Spot have no tag entry at all, Steely Spirit is `untagged`	0 on this corpus; they are doubles abilities and the corpus is doubles
Punching Glove	`onBasePower`	absent	`isNonstandard: 'Past'` — **banned in this format.** Recorded so nobody wires it
the 17 plates, the orbs, Soul Dew	`onBasePower`	absent	all `isNonstandard: 'Past'` — banned. The type-item cousins in 2a are the legal ones
Collision Course / Electro Drift / Brine / Retaliate	move `onBasePower`	not in `MC.moves` at all — a move-table gap, not a stage gap	filed for whoever owns `build_engine_data.js`

2c. SAME STAGE — checked, and correct

This list exists so the next session does not re-audit it. Each was measured, not read.

multiplier	authority event	ours	evidence
Life Orb x1.3	`onModifyDamage` `[5324,4096]`	`:2411-2412` , folded into `mod` via `ch4096`	**3.9% (11/284)** against a control of 4.1% — the same rows
Multiscale / Shadow Shield x0.5 from full	`onSourceModifyDamage`	`:2437-2451` `damageReduce`	**4.1% (12/294)** — identical to the control
Tinted Lens x2 on resisted	`onModifyDamage`	`:2453`	**4.1% (12/294)** — identical to the control
Filter / Solid Rock / Prism Armor x0.75 on SE	`onSourceModifyDamage`	`:2437-2451`	single rows agree exactly
Ice Scales x0.5 special	`onSourceModifyDamage` (**not** a stat modifier, which is the natural mis-statement)	`:2437-2451`	Alakazam Psychic -> Snorlax 36 vs 36
Punk Rock defensive x0.5 sound	`onSourceModifyDamage`	`:2437-2451`	Hyper Voice -> Snorlax 24 vs 24 (control 49)
Expert Belt x1.2 on SE	`onModifyDamage` `[4915,4096]`	`:2502-2503`	agrees
the resist berries x0.5	`onSourceModifyDamage`	`:2514-2515`	Chople on a SE Fighting hit, 114 vs 114
Neuroforce x1.25 on SE	`onModifyDamage`	`:2452`	agrees. (`neuroforce` is absent from `data/tags.json` and is name-wired — correct today, brittle)

multiplier	authority event	ours	evidence
Reflect / Light Screen / Aurora Veil `[2732,4096]` in doubles	`onAnyModifyDamage`	`:2462-2466` `DOUBLES_SCREEN = 2732/4096`	constant matches the authority's doubles branch exactly
the four Ruin abilities x0.75	`onAnyModifyDef` / `onAnyModifyAtk` / `onAnyModifySpA` / `onAnyModifySpD` — **STAT** stage	`:2242-2245` `md4096` on A or D	Sword of Ruin 81 vs 81, Tablets of Ruin 46 vs 46
sand Rock SpD x1.5 / snow Ice Def x1.5	`onModifySpD` / `onModifyDef` using `this.modify`	`:2208-2209` `md4096(D, 1.5)`	correct event, correct arithmetic
Huge Power / Pure Power x2, **Guts** x1.5, **Solar Power**, **Orichalcum Pulse**, **Hadron Engine** `[5461,4096]`	`onModifyAtk` / `onModifySpA`	`:2225-2230`	Huge Power row agrees (clamped, but the pre-clamp ratio is exact)
Choice Band / Choice Specs / Assault Vest x1.5	`onModifyAtk` / `onModifySpA` / `onModifySpD`	`:2192-2194`	right stage — **and all three are** `isNonstandard: 'Past'` **, so they are dead code in this format**
Adaptability x2 STAB	`onModifySTAB`	`:2391-2392` `stabBoost`	132 vs 132
spread x0.75	`modify` at `:1737`	`:2247`	13/282, the control's own rows
burn x0.5	`modify` at `:1818`	`:2531`	5.0% against a 4.6% control
Facade's burn exemption	`move.id !== 'facade'` at `:1817`	`:2405` keyed on `conditionalPower.when === 'userStatused'`	shape-keyed, membership printed, exactly one move
Technician's <=60 gate	`this.modify(bp, this.event.modifier)` at `:1650`	`:2308` gates on the raw `mvBP`	**EQUIVALENT, and this was nearly filed as a bug.** Technician's `onBasePowerPriority` is **30, the highest in the format**, so `event.modifier` is still 1 when its gate runs and `modify(bp, 1) === bp` . Proved: Body Slam (85 BP) + Technician + Silk Scarf gets no Technician boost in either engine
the randomizer's POSITION	`:1755`	`:2528`	both endpoints match on every control row
the certain crit's position	`:1751`	`:2276-2281`	Frost Breath and Storm Throw agree at BOTH endpoints
the base-damage formula itself	`:1718`	`:2246`	a 294-row control at 4.1% residual, and the 12 failures are named moves (Beak Blast, Night Daze, Spirit Shackle, Trop Kick, Fickle Beam, Apple Acid), not arithmetic

3. THE TWO NON-MODIFIERS, CHECKED EXPLICITLY

The crit: our arithmetic is right, our POSITION is wrong in the battle loop

- **Is it a plain truncated x1.5, or did it go through the 4096ths helper?** Plain. `:2280` `Math.floor(base*1.5*critMult)` and `:5917` `Math.floor(dmg*1.5*critMult)` . Neither touches `md4096` . That matches the authority's `tr(baseDamage * 1.5)` and its "crit - not a modifier" comment. **Correct.**
- **Position.** The authority puts the crit at `:1751` — *before* the randomizer, STAB, the type chart, burn and the ModifyDamage chain. `dmgRange` puts its certain crit in exactly that place (`:2276` , after spread and weather, before `roll()`) and it passes at both endpoints. The **battle loop's rolled crit** (`:5912-5918`) multiplies the number that has already been rolled, STAB'd, type-charted, burnt and chain-spent.

At the TOP roll the randomizer is the identity, so the error is smaller and the wrongness looks like nothing. Measured over random triples:

arm	disagree
no crit, bottom roll (**the control**)	20/364 — **5.5%**
crit, top roll	72/346 — 20.8%
crit, bottom roll	165/355 — **46.5%**
crit + Life Orb, bottom roll	210/340 — **61.8%**
crit + Sniper, top roll	109/313 — 34.8%
crit + Sniper, bottom roll	179/331 — 54.1%

Sniper compounds it twice: it is `onModifyDamage` , so it belongs in the final chain, and ours folds it into the crit's plain multiply.

The roll: same position, different shape

- **Position: SAME.** `:2528` `Math.floor(base * r / 100)` sits between the crit and STAB, which is where `battle.ts:2388` `randomizer` sits. Confirmed at both endpoints on every control row.
- **Shape: different, and already documented** in `engine/game_differential.js` 's header — 11 uniform integers here against 16 inverted indices there, agreeing only at the endpoints. **That note does not cover position, and position is fine.** The shape question is not this audit's.

The crit, third road: the DELAYED payout took no draw at all — added 5.241.0

The two roads above are about POSITION. This one was about ABSENCE, and absence is the harder thing for this document to have caught, because a stage that is never entered produces no wrong number to compare — it produces the same number every time.

The authority hands a delayed payout to `trySpreadMoveHit` (`data/conditions.ts:415`), so it walks the same step list as a direct click: the crit step at `sim/battle-actions.ts:1156` → `:1636-1642` , `critMult = [0,24,8,2,1]` at `:1633` , the plain ×1.5 at `data/mods/champions/scripts.ts:222` and the `|-crit|` line at `:285` . Champions carries no `futuremove` key, so mainline is the authority and that is derived rather than assumed.

How it was measured, and why a sampled run could never have found it. The die was pinned, not sampled — crit-CERTAIN on one arm and crit-IMPOSSIBLE on the other, same board, same attacker:

die	authority	ours, before		
crit-certain	**72**, `\`	-crit\	` present	**69**, no line

die	authority	ours, before
crit-impossible	**48**, no line	**69**, no line

Identical output across a varied knob is the finding. The direct-click control on the same attacker moved on both engines, so the harness could see a crit either way.

Where the fix lands, in this document's terms. The payout is RE-PRICED as a certain crit through `dmgRange` — the road §3 above measures as being at the authority's position and passing at both endpoints — rather than multiplied afterwards like the battle loop's rolled crit. A late ×1.5 would have produced a number that is wrong in exactly the way the 46.5% bottom-roll row above is wrong. The `-crit` line is written between the effectiveness line and the damage line, which is the authority's order.

The rolled-crit defect in the battle loop is UNCHANGED by this, and the table above it still stands. This adds a road; it does not fix that one.

---

4. THE CHAIN, NOT ONLY THE STAGE — FRIEND GUARD

Friend Guard is `onAnyModifyDamage`, so it belongs in the **same chain** as Life Orb, the screens, the resist berries and Expert Belt. Ours applies it at `:5896` as its own `md4096` on the number `dmgRange` has already spent its chain on.

Right stage, wrong chain, and it costs a point on a fifth of values. Two spends against one, over base damages 20..300:

```
Life Orb x1.3 then Friend Guard x0.75
  authority  modify(d, chain(chain(1, 1.3), 0.75))    one spend
  ours       modify(modify(d, 1.3), 0.75)             two spends
  -> 60 of 281 base-damage values disagree (21.4%);  d=45: authority 44, ours 43
```

Friend Guard is **1015 sheets**. Helping Hand (§2a) has the same double-spend problem on top of its stage problem.

---

5. WHAT THIS AUDIT COULD NOT SEE, SAID OUT LOUD

The harness calls `battle.actions.moveHit`, which is one level below `spreadMoveHit`. Three events never fire there, so three things are argued from the handler source rather than measured:

- `ModifyMove` — so Sheer Force's `hasSheerForce` and the -ate abilities' `typeChangerBoosted` are never set by Showdown. Sheer Force was staged by hand; **the -ate abilities could not be** and their row above is reasoned from `data/abilities.ts`, not measured.
- `spreadHit` and `willCrit` — both staged by hand, and the staging is stated in the probe.
- **the hit loop**, so `move.hit` and multi-hit are out of scope. `tests/test-engine-diff.js` already records this boundary.

Two further limits, stated because they are the shape of the control failures this project keeps finding:

- **The reference clamps at the defender's HP and we do not.** Four rows in the first pass read as disagreements purely because Showdown had KO'd. Every rate above drops those rows.
- **Both abilities are set explicitly on both sides, always.** The first pass left the defender at its species default and measured Araquanid's own Water Bubble against a blank, and Heliolisk's own Dry Skin against a blank. That is the Choice-Scarf-against-a-Choice-Scarf failure, and it happened here before it was caught.

---

6. THE ORDER TO FIX IN

1. **Fairy Aura / Dark Aura at the BasePower stage, with `[5448,4096]` and `[3072,4096]`.** Another agent is wiring `auraBoost` now. §0 is the acceptance test: 8/8, not 2/8.
2. **The 18 type items** (the largest class in 2a) — one line, `damageMultType` moves from `MODMUL` to a base-power chain. Everything below shares that chain, so build it once.
3. **Helping Hand** (4306 clicks) — and it needs the chain as well as the stage.
4. **Technician, Tough Claws, Sharpness, Sheer Force, Iron Fist, Mega Launcher, Strong Jaw, Punk Rock, Supreme Overlord, Expanding Force / Rising Voltage, Muscle Band, Wise Glasses, Dry Skin** — the same move into the same chain.
5. **Thick Fat / Heatproof / Purifying Salt / Water Bubble** into the STAT stage, beside the Ruin abilities that already live there.
6. **The battle loop's rolled crit** — it must be applied inside `dmgRange`, before the roll, not to `dmgRange`'s output. This is the largest single measured effect in the document (46.5% -> 61.8%).
7. **Friend Guard** into the ModifyDamage chain rather than beside it.
8. **The field terrain multipliers**, which are absent entirely.
9. ~~`damageBoost` — 44 abilities carry it and nothing reads it.~~ **DONE, in two passes.** ROADMAP #92 (3.7x) wired the unconditional, type-naming half — five abilities, all 0 corpus uses. ROADMAP #112 (3.83.0) made the HP condition machine-readable and added the other four: Blaze, Torrent, Overgrow, Swarm — 9,141 uses. Solar Power is still refused, correctly: its condition is a WEATHER and `inWeather` is a separate clause. The membership was printed before either wiring, and `tests/test-damage-stages.js` re-derives it every run.

Every one of these needs a failing probe in `tests/test-mechanics.js` first. None of them is open work until it has one.